

Multi-level lot-sizing and job shop scheduling with lot-streaming: Reformulation and solution approaches[☆]

Mohammad Rohaninejad^{*}, Zdeněk Hanzálek

Industrial Informatics Department, Czech Institute of Informatics Robotics and Cybernetics, Czech Technical University in Prague, Czech Republic

ARTICLE INFO

Keywords:

Lot-sizing
Job shop scheduling
Constraint programming
Heuristic
Fix and relax

ABSTRACT

This paper addresses a multi-level lot-sizing and job shop scheduling problem with lot-streaming. In the multi-level production system, workstations receive materials from the lower level, and after some operation, materials are shipped to the next level. Hence, establishing a material balance between the different levels is the most challenging part of multi-level production planning and scheduling. The material balance can be performed with or without lot-streaming. Lot-streaming effectively enables consecutive operations to overlap by splitting a processing lot into several sub-lots. In small-bucket time models, this capability is taken into account by establishing the material balance in each small unit of time (micro-period), which makes the models computationally expensive. In the present work, a novel and much less complicated big-bucket time formulation has been developed, which incorporates lot-streaming considering sequence-dependent setup times and capacitated machines. Computational experiments affirm the promising results of the proposed model compared to the well-known models in the literature. Moreover, two efficient heuristics have been developed for solving larger-size problems. First, the fix-and-relax algorithm as a constructive heuristic is combined with the fix-and-optimize algorithm as an improvement heuristic. Next, a decomposition heuristic is proposed using mixed-integer programming (MIP) and constraint programming (CP) in the master and sub-problem, respectively. The computational results show that the proposed heuristics are very efficient, even in solving large-sized problems.

1. Introduction

In recent years, under fierce competition in the market, companies have been exploring more precise and realistic production plans to increase their productivity and gain an edge in the marketplace. Accordingly, there has been increasing interest shown in the simultaneous lot-sizing and scheduling problem as one of the most challenging problems in production planning with vast application areas, such as the automotive industry (Yan et al., 2003), printed circuit board assembly (Qin et al., 2019), mold-injection (Ríos-Solís et al., 2020), food processing (Claassen et al., 2016), consumer electronics, containers, and packaging materials (Li et al., 2016). This problem consists of medium and short-term decisions that belong to two different planning levels. In the medium-term planning levels, the number of items to be produced (lot-sizing) is determined during the planning horizon to fulfill strategic goals (Carvalho and Nascimento, 2021). In the short-term planning levels, the sequence of tasks within periods is determined to meet specific operational and short-term objectives (Rohaninejad et al., 2016).

Research on lot-sizing started with the classical economic order or production quantity (EOQ, EPQ) models. The majority of these models were concerned with defining a total cost function from an economic point of view (Andriolo et al., 2014). The general idea is to find the ideal quantity of an item that should be ordered or produced to satisfy the customer demand while minimizing the total system costs, including ordering costs (setup costs in the EPQ models) and inventory costs. A higher order quantity results in higher inventory holding costs, and a lower order quantity increases the order/setup costs. In summary, these models find the optimal order/production quantity by making a trade-off between the order/setup cost and inventory holding cost. The scheduling decisions are not integrated into classical lot-sizing problems. Therefore, the usual approach is to solve the lot-sizing problem first and then solve the scheduling problem separately (Drexel and Kimms, 1997). This approach often leads to an infeasible or sub-optimal solution when considering machine availability constraints, sequence-dependent setup times, some

[☆] The research has been supported by the Ministry of Education, Youth and Sports within the dedicated program ERC CZ under the project POSTMAN with reference LL1902 and the Grant Agency of the Czech Republic under the Project GACR 22-31670S.

^{*} Correspondence to: 166 36 Prague 6, Czech Republic.

E-mail address: Mohammad.Rohani.Nezhad@cvut.cz (M. Rohaninejad).

precedence constraints between operations, and sequence-dependent setup cost (Carvalho and Nascimento, 2021). Therefore, establishing an integrated lot-sizing and scheduling problem (ILSP) can provide more realistic and applicable decision-making.

Material balance is the most challenging part of the multi-level ILSP, where workstations receive materials from the lower level and, after some operation, the materials are shipped to the next level. This issue is essential in view of the fact that it directly affects the master and detailed scheduling of jobs in production planning. In master scheduling, the flow of materials between two periods is determined by holding and moving inventory from one period to the next. Moreover, in detailed scheduling, the start and finish times of operations within a period must be synchronized with the material balance constraints. Batching and lot-streaming are two approaches to establish the material balance in the multi-level ILSP (Almeder et al., 2015). In the batching assumption, which encompasses most of the existing studies, it is assumed that the successor items cannot start until the predecessor items are finished. The inability to start successor items before the end of their predecessor items reduces the throughput of the system, while relaxing it leads to time-saving and more realistic scheduling through reducing the machines' idle time. In lot-streaming, overlapping the production of predecessor and successor items is possible by splitting a processing lot of predecessor items into several sub-lots and gradually transitioning them to the next stage (successor items).

The studied models in the literature of multi-level ILSP, are divided into small-bucket time (SBT) and big-bucket time (BBT) formulations. SBT models are based on subdividing the periods of classical lot-sizing problems into several micro-periods, where the number of items produced in each micro-period is limited. The discrete lot-sizing and scheduling problem (DLSP) presented by Fleischmann (1990) is the first study that integrates lot-sizing and scheduling through the SBT model. The continuous setup lot-sizing problem (CLSP) (Hindi, 1995) and proportional lot-sizing and scheduling problem (PLSP) (Kimms and Drexel, 1998) are other types of SBT models introduced later. Compared to DLSP, CLSP is a step toward more realistic situations. In DLSP, there is a fundamental assumption called "all-or-nothing" which assumes that either only one item can be produced per micro-period using the full capacity of the period or nothing. In CLSP, this assumption is given up, and any production quantity can be produced in a micro-period, while only one item can still be produced per period. PLSP addresses the shortcoming of CLSP by allowing the remaining capacity of the micro-period to be used with a second item. For a detailed review of these models and their differences, refer to Drexel and Kimms (1997). In this type of formulation, the assignment of tasks to micro-periods determines their sequence according to the natural order of the micro-periods. Therefore, the material balance can be enforced between two consecutive micro-periods, which naturally allows for lot-streaming. Although the natural scheduling of tasks and straightforward lot-streaming are recognized as advantages of these models, their performance strongly depends on the micro-period length. Since periods in SBT models usually correspond to small time slots, such as hours or shifts, the complexity of the model is drastically increased. This is because all the variables (such as production quantity, operations setup, and inventory variables) and constraints (such as the material balance constraint) that correspond to periods in the classical lot-sizing problem must be defined for each micro-period. In addition, problems with long setup times cannot be modeled if the length of the micro-periods is too short (Wan and Zhan, 2021). Conversely, decreasing the schedule precision and increasing the machines' wasted capacity are the consequences of micro-periods with long lengths. Therefore, numerous studies have been conducted on the BBT formulation of ILSP to take a step toward real-world size problems (Haase, 1996; Fleischmann and Meyr, 1997; Rohaninejad et al., 2015; Ramezani et al., 2013c). In contrast to the small bucket time model, where one or, at most, two items can be produced in one period, the periods in the big bucket time formulation are long enough where many different items can be

produced with the same resource in one period (Drexel and Kimms, 1997). Haase (1996) and Fleischmann and Meyr (1997) developed capacitated lot-sizing with sequence-dependent setup costs (CLSD) and the general lot-sizing and scheduling problem (GLSP) as the first BBT models, respectively. However, their models were not developed for the multi-level ILSP. Hence, several studies were later conducted to develop the BBT models for the multi-level ILSP (Rohaninejad et al., 2015; Ramezani et al., 2013c; Karimi-Nasab and Seyedhoseini, 2013).

This paper deals with multi-level, multi-machine capacitated lot-sizing and scheduling in the job shop environment. A new and efficient BBT formulation considering lot-streaming has been developed to address the shortcomings of the existing models. Two iterative and hierarchical solution methods have been proposed to find near-optimal solutions for large instances. As an iterative approach, different variants of the fix-and-relax and fix-and-optimize algorithms have been proposed and combined to solve the problem. Moreover, as a hierarchical approach, an efficient decomposition heuristic based on mixed-integer programming and constraint programming (MIP/CP) has been developed using a feasibility cut to deal with infeasible solutions.

The rest of this paper is organized as follows: Section 2 reviews the most relevant works in the literature and the contribution of this paper. The problem description, assumptions, and a novel big-bucket time model are presented in Sections 3 and 4. Section 5 illustrates the proposed solution methods. Section 6 presents the computational results to validate the proposed model and evaluate the solution methods. Some managerial insights are provided in Section 7. Finally, Section 8 presents conclusions and directions for future studies.

2. Literature review

The initial studies on integrated lot-sizing and scheduling problems were mostly focused on single-level structures. Several single-level SBT (Fleischmann, 1994; Kaczmarczyk, 2011; Almada-Lobo et al., 2007) and BBT (Meyr, 2000; Xiao et al., 2013; Lang and Shen, 2011; Xiao et al., 2015) models have been developed so far in single machine (Fleischmann, 1994; Almada-Lobo et al., 2007; Meyr, 2000; Lang and Shen, 2011) or parallel machine (Kaczmarczyk, 2011; Xiao et al., 2013, 2015) settings. With the growing complexity of production environments, studying multi-level problems, such as the flow shop (Qin et al., 2019; Ramezani et al., 2013c; Babaei et al., 2014), job shop (Karimi-Nasab and Seyedhoseini, 2013; Giglio et al., 2017; Poursabzi et al., 2018), flexible flow shop (Wan and Zhan, 2021; Mahdiah et al., 2011), or flexible job shop (Rohaninejad et al., 2015), has attracted research interest. The last decade should be looked at more to review the literature on the multi-level ILSP while demonstrating the development path for the multi-level models. As initial works, multi-level ILSP was documented by Kimms (1999, 2012) and Grünert (1998). Following the SBT formulation, Kimms (1999) and Kimms (2012) has expanded the PLSP models for multi-level and multi-machine ILSP, considering the Gozinto factor between different levels. Grünert (1998) also presented a multi-level BBT model while different levels were synchronized by applying the inventory balancing constraints during a period.

In contrast to the classical lot-sizing problem, in which inventory balance is enforced at the beginning or at the end of periods, in the multi-level ILSP, inventory balancing must be established throughout a period. Accordingly, the synchronization of the different levels or stages is the most critical part of the multi-level ILSP, so that the starting time of the tasks should be determined to avoid inventory shortages within a period. Regardless of the general assumptions, studies conducted on the multi-level ILSP can be distinguished based on (i) the type of SBT or BBT models and also (ii) their synchronization mechanism.

As mentioned earlier, modeling the multi-level ILSP in the SBT mode is straightforward due to the transfer of the material balance from the macro-period to the micro-period. Hence, numerous studies have been performed for multi-level ILSP in different types of SBT,

such as DLSP (Brüggemann and Jahnke, 1994; Göthe-Lundgren et al., 2002; Muramatsu et al., 2003), CLSP (de Araujo et al., 2007; Baldo et al., 2014), and PLSP (Kimms, 1996; Stadtler, 2011; Stadtler and Sahling, 2013). Although synchronization within micro-periods has yet to be addressed in most studies, some have considered it. Stadtler (2011) extended the single-level, single-machine PLSP to a multi-level PLSP with three types of activity in each micro-period consisting of the production at the beginning of the period, production at the end of the period, and the production setup. He considered the batching approach for synchronization so that items produced at the beginning of the micro-period allow their successor item to be produced at the end of the same micro-period.

Despite the advantages of the SBT approach for modeling the multi-level ILSP, their drawbacks, especially their high computational complexity, cannot be overlooked. Hence, the BBT approach has received great deal attention in different types of multi-level GLSP (Rohaninejad et al., 2015; Babaei et al., 2014; Fandel and Stammen-Hegene, 2006; Ramezani et al., 2013a; Babaei et al., 2012), multi-level CLSD (Grünert, 1998; Almeder et al., 2015; Clark, 2003), and multi-level continuous time lot-sizing and scheduling problem (CTLSP) (Ramezani et al., 2013c; Karimi-Nasab and Seydihoseini, 2013; Almeder et al., 2015; Dauzère-Pérès and Lasserre, 2002; Ramezani et al., 2013b; Karimi-Nasab et al., 2015; Wolosewicz et al., 2015; Gomez Urrutia et al., 2014; Filho et al., 2012). Due to the simplicity of the modeling, batching synchronization is the dominant approach observed in the literature of the multi-level BBT models. Rohaninejad et al. (2015), and Almeder et al. (2015) are among the few works that have considered lot-streaming in their multi-level BBT models. They seek to ensure that, for every time point τ in a period, the inventory of the predecessor operation plus its production until τ is not less than its consumption during the production of the successor operation.

Rohaninejad et al. (2015) developed a lower bound for the inventory of the predecessor item at the beginning of the successor operation to prevent its interruption during its processing time. For multi-level CLSD models, Almeder et al. (2015) also developed a lot-streaming synchronization. Their synchronization constraints lead to a nonlinear model allowing the immediate use of the produced predecessor items and maintaining a non-negative inventory within the period. However, they have to introduce some additional variables and constraints for the linearization of the model. Seeanner and Meyr (2013) and Seeanner et al. (2013) presented a new GLSP model for multi-level ILSP in which the length of the micro-periods depends on the synchronization constraints. Some additional constraints and variables (e.g., the machine idle time) were introduced in Seeanner and Meyr (2013) so that, in an identical micro-period, the successor item must neither starts before the production of the predecessor item starts nor finishes before the production of the predecessor item ends. Moreover, Seeanner et al. (2013) determined a minimum value for the length of micro-periods based on the processing time of the slower operation. The variability in the length of the micro-periods caused them to introduce a variable representing the machine's idle time and the machine standby cost which imposes an additional computational cost. Their modeling shows a type of lot-streaming is taken into account. To the best of our knowledge, there is no CTLSP model with lot-streaming synchronization. Ramezani et al. (2013a) presented a CTLSP model for capacitated multi-stage production systems where overlapping operations on two consecutive machines are possible. They considered the permissible overlap as a parameter of the model, where it should be a function of the other decision variables of the model.

This paper develops an efficient big-bucket MILP formulation for multi-level lot-sizing and scheduling in the job shop problem, which can be easily generalized to other production environments. The main advantages of the proposed model are: (i) providing higher quality solutions, (ii) higher feasibility and a re-schedulability indicator, (iii) higher responsiveness, (iv) lower work-in-process and machine idle times, and (v) lower computational complexity. The model was applied to several

instances, and its differences from some well-known models in the literature were analyzed. Moreover, various fix-and-relax heuristics are presented in combination with a fix-and-optimize algorithm, and an efficient decomposition heuristic is also developed for the larger-sized instances.

3. Problem description

This paper investigates the simultaneous lot-sizing and job shop scheduling considering sequence dependent-setup time and machine capacity constraints. The scheduling horizon consists of T periods ($t = 1, \dots, T$) while several products can be produced in each period. There is a predefined number of jobs ($j = 1, \dots, J$), each including several operations ($O_{j,o} \mid o = 1, \dots, h_j$) processed by M capacitated machines with deterministic processing times. Each job has a predefined amount of demand ($D_{j,t}$) in each period that must be met at the end of the period. Back-orders, i.e., fulfilling demands after their due date, are not allowed. The decision-maker must determine the number of setups, lot-sizes (the amount of production in each setup), the sequence, and the start time of operations so that all the demands meet their due dates and the sum of the setup, production, and inventory holding costs is minimized. Since the cost-based objective function alone is not sufficient to examine the effect of lot-streaming on the ILSP, another type of time-based objective function is presented further in Section 6.2.1.4. In this model, the back-order is permitted while the decision-maker should determine the decision variables to minimize the total weighted tardiness of the jobs.

As mentioned, the overlap between succeeding operations of a job processed at different workstations is allowed. Fig. 1 shows the different situations possible within a period. These situations for period t and operation $O_{j,(o+1)}$ depend on (i) the inventory of the predecessor operation ($O_{j,o}$) available at the end of the previous period (variable $I_{j,o,t-1}$) and (ii) how much the operation $O_{j,(o+1)}$ is supposed to be produced in period t (variable $x_{j,(o+1),t}$). The successor operation can end before the predecessor operation starts (Fig. 1(a)) if there is a sufficient inventory of the predecessor operation at the end of the previous period. The successor operation can start before the predecessor operation (Figs. 1(b) and 1(c)) if the inventory of the predecessor operation at the end of the previous period can partially cover the production of the successor operation in the current period. Other situations (Fig. 1(d) to 1(f)) can occur even if there is no inventory for the predecessor operation at the end of the previous period. Hence, we do not have to wait for the predecessor operation to end to start the successor operation. The produced units of the predecessor operation related to one specific setup can be partially delivered to its successor before the predecessor operation's end.

Further assumptions are as follows.

- Machines are available all the time.
- The total processing time of the machines within a period is capacitated.
- Machines can process, at most, one operation at a time.
- The setup for an operation can be performed only once a period.
- The due dates of all the orders are at the end of each period.
- The setup, production, and inventory holding costs may vary during the planning horizon.
- There is no lead time (e.g., time for transportation, inspection, cooling, etc.) between two consecutive operations of an identical job.
- Holding inventory is permitted for in-process products at the end of each period.
- The triangle inequality holds for sequence-dependent setup times.
- The machines' setup state conservation across periods by the setup carryover or crossover is ignored.

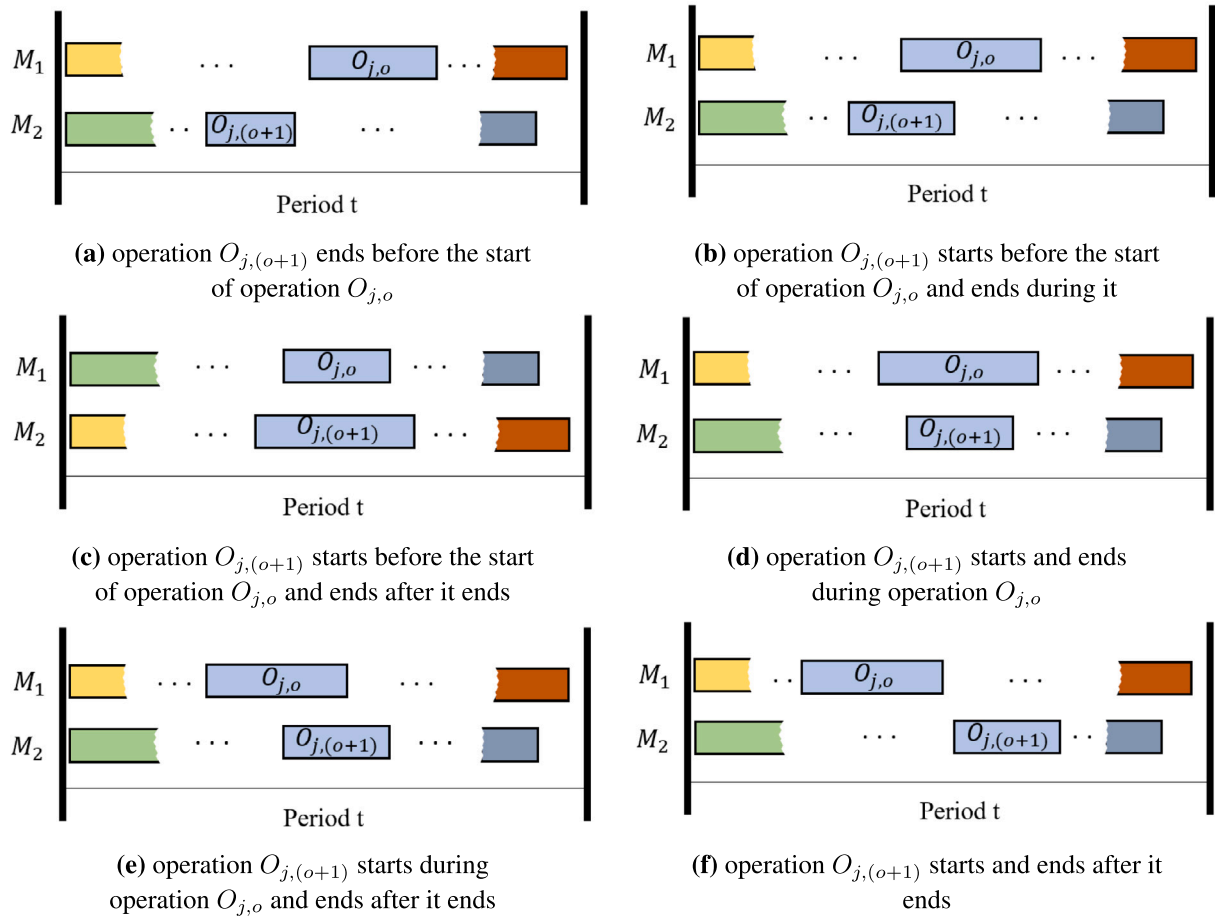


Fig. 1. Possible schedules of two consecutive operations of the same job.

4. A mixed integer programming formulation

This section presents the MILP formulation of the capacitated lot-sizing and job shop scheduling. The proposed model can be applied to a series structure of jobs (each operation has exactly one predecessor and one successor, except the first and last operations) and convergent structure (each operation has at least one predecessor and exactly one successor, except the first and last operations). However, the application of the model in the divergent structure (each operation has exactly one predecessor and at least one successor, except the first and last operations) requires some modifications that are explained in Section 4.1. We define the following notation.

Indices

j, j', j''	Index of jobs ($j, j', j'' = 1, \dots, J$)
o, o', o''	Index of operations
h_j	Last operation of job j
m	Index of machines ($m = 1, \dots, M$)
t	Index of periods ($t = 1, \dots, T$)

Parameters

$D_{j,t}$	Demand of job j at the end of period t
$p_{j,o}$	Processing time needed to produce one unit of product related to operation $O_{j,o}$
$C_{m,t}$	Capacity of machine m during period t

$\delta_{j,o,j',o'}$	Setup time of $O_{j',o'}$ if processed immediately after $O_{j,o}$ on the same machine
$\delta_{0,j,o}$	Setup time of $O_{j',o'}$ if it is the first operation produced on a machine in the period
$a_{j,o}$	Machine that is assigned to process $O_{j,o}$
$sc_{j,o,t}$	Setup cost needed to run $O_{j,o}$ in period t
$pc_{j,o,t}$	Production cost needed to produce one unit of product related to $O_{j,o}$ in period t
$hc_{j,o,t}$	Inventory holding cost for one unit of product related to $O_{j,o}$ at the end of period t
$Q_{j,o}$	Set of predecessor operations for $O_{j,o}$
$\gamma_{j,o',o}$	Gozinto factor; consumption coefficient of $O_{j,o'}$ in $O_{j,o}$ while ($O_{j,o'} \in Q_{j,o}$)
L	Length of each period
G	A big positive number

Variables

$x_{j,o,t}$	Quantity of $O_{j,o}$ that is produced in period t (lot-size)
$I_{j,o,t}$	Amount of inventory related to $O_{j,o}$ at the end of period t
$s_{j,o,t}$	Start time of $O_{j,o}$ in period t
$z_{j,o,t}$	1 iff $O_{j,o}$ is produced in period t ; 0, otherwise
$w_{j,o,t}$	1 iff there is not enough inventory of predecessor operation $O_{j,o'}$ at the end of period t to completely cover production of $O_{j,o}$ in period t ($I_{j,o',t-1} < \gamma_{j,o',o} \cdot x_{j,o,t}$); 0, otherwise
$y_{j,o,j',o',t}$	1 iff $O_{j',o'}$ produced after $O_{j,o}$ in period t on the same machine; 0, otherwise
TSC	Total system cost

With the notation introduced above, the big-bucket time formulation of the problem is as follows:

$$\text{Min } TSC = \left(\sum_{t=1}^T \sum_{j=1}^J \sum_{o=1}^{h_j} z_{j,o,t} \cdot sc_{j,o,t} + x_{j,o,t} \cdot pc_{j,o,t} + I_{j,o,t} \cdot hc_{j,o,t} \right) \quad (1)$$

$$I_{j,h_{j,t}-1} + x_{j,h_{j,t}} - D_{j,t} = I_{j,h_{j,t}} \quad \forall t, \forall j \quad (2)$$

$$I_{j,o,t-1} + x_{j,o,t} - \gamma_{j,o,o'} \cdot x_{j,o',t} = I_{j,o,t} \quad \forall t, \forall (j, o, o') \mid O_{j,o} \in Q_{j,o'} \quad (3)$$

$$x_{j,o,t} - z_{j,o,t} \cdot G \leq 0 \quad \forall t, \forall (j, o) \quad (4)$$

$$\sum_{j=1}^J \sum_{o=1}^{h_j} x_{j,o,t} \cdot p_{j,o} \leq C_{m,t} \quad \forall t, \forall m \quad (5)$$

$$I_{j,o,t-1} - \gamma_{j,o,o'} \cdot x_{j,o',t} + w_{j,o,t} \cdot G \geq 0 \quad \forall t, \forall (j, o, o') \mid O_{j,o'} \in Q_{j,o}; \& I_{j,o',0} = 0 \quad (6)$$

$$s_{j,o,t} + \left(\gamma_{j,o,j',o',t} + (2 - z_{j,o,t} - z_{j',o',t}) \right) \cdot G \geq s_{j',o',t} + \delta_{j',o',j,o} + x_{j',o',t} \cdot p_{j',o'} \quad (7)$$

$$\forall t, \forall (j, o), \forall (j', o') \mid (j, o) \neq (j', o') \& a_{j,o} = a_{j',o'}$$

$$s_{j',o',t} + \left((1 - \gamma_{j,o,j',o',t}) + (2 - z_{j,o,t} - z_{j',o',t}) \right) \cdot G \geq s_{j,o,t} + \delta_{j,o,j',o'} + x_{j,o,t} \cdot p_{j,o} \quad (8)$$

$$\forall t, \forall (j, o), \forall (j', o') \mid (j, o) \neq (j', o') \& a_{j,o} = a_{j',o'}$$

$$s_{j,o,t} \geq t \cdot L + \delta_{0,j,o} \cdot z_{j,o,t} \quad \forall t, \forall (j, o) \quad (9)$$

$$s_{j,o,t} + x_{j,o,t} \cdot p_{j,o} \leq (t+1) \cdot L \quad \forall t, \forall (j, o) \quad (10)$$

$$s_{j,o,t} + (1 - w_{j,o,t}) \cdot G \geq s_{j,o',t} + \gamma_{j,o',o} \cdot p_{j,o'} - \left\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \right\rfloor \cdot p_{j,o} \quad (11)$$

$$\forall t, \forall (j, o, o') \mid O_{j,o'} \in Q_{j,o}; \& \gamma_{j,o',o} \cdot p_{j,o'} \leq p_{j,o} \& I_{j,o',0} = 0$$

$$s_{j,o,t} + (1 - w_{j,o,t}) \cdot G \geq s_{j,o',t} + (\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot \left(x_{j,o,t} - \left\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \right\rfloor \right) + (1 - \left\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \right\rfloor) \cdot p_{j,o} \quad (12)$$

$$\forall t, \forall (j, o, o') \mid O_{j,o'} \in Q_{j,o}; \& \gamma_{j,o',o} \cdot p_{j,o'} > p_{j,o} \& I_{j,o',0} = 0$$

$$x_{j,o,t}, I_{j,o,t}, s_{j,o,t} \in \mathbb{R}^+; z_{j,o,t}, w_{j,o,t}, \gamma_{j,o,j',o',t} \in \{0, 1\} \quad \forall t, \forall (j, o), \forall (j', o') \quad (13)$$

Eq. (1) defines the objective function comprising the sum of the setup, production, and inventory holding costs. The inventory balance regarding the last operation of each job, between periods, is ensured by Constraint (2). Constraint (3) enforces the inventory balance for other operations. Constraint (4) indicates that the setup regarding an operation must be performed before starting that operation.

The limited capacity of each machine is enforced by Constraint (5). Constraint (6) is used to detect whether there is enough inventory of the predecessor operations in the previous period to meet the production of the successor operation in the current period. This constraint is required to determine the precedence or overlap between two consecutive operations. A successor operation ($O_{j,o}$) can start and end before its predecessor operation ($O_{j,o'}$) in period t if $w_{j,o,t} = 0$.

Constraints (7) and (8) are the disjunctive constraints that are well-known constraints in scheduling theory (Castro and Grossmann, 2012). These constraints enforce the precedence ordering for any two

operations $O_{j,o}$ and $O_{j',o'}$ in period t , while they use a shared resource (machine). These constraints must hold if both operations are produced in period t .

Constraints (9) and (10) assure that the start and finish time of the production of all the operations in a given period t should occur during the start time and finish time of period t . Constraint (9) is always a valid inequality due to the triangle inequality rule for the sequence-dependent setup time.

Constraints (11) and (12) are the precedence constraints of two consecutive operations of the same job considering lot-streaming. Constraint (11) guarantees the minimum required delay to start the successor operation after the start of its predecessor, while a predecessor operation is produced faster than its successor. In that case, the production of the successor only has to wait for the production of one unit of the predecessor operation to start the production of its given lot-size. This can even be shifted ($\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \rfloor \cdot p_{j,o}$) units to the left by using the leftover inventory of the predecessor operation from the previous period (period $t-1$). Constraint (12) deals with the same situation while the predecessor operation is slower than its successor. The validity of this constraint is proven by Theorem 4.1.

Theorem 4.1. For the uninterrupted production of $O_{j,o}$ in period t , the minimum required delay to start $O_{j,o}$ after the start time of $O_{j,o'}$ ($de_{j,o,t}$) is given by:

$$de_{j,o,t} = (\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot \left(x_{j,o,t} - \left\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \right\rfloor \right) + (1 - \left\lfloor \frac{I_{j,o',t-1}}{\gamma_{j,o',o}} \right\rfloor) \cdot p_{j,o} \quad (14)$$

Where:

$$\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} < 1$$

Proof. To prove it, first, we assume $I_{j,o',t-1}$ is equal to zero, and every fraction of the output of $O_{j,o'}$ can be delivered to $O_{j,o}$ to be processed (continuous mode). Fig. 2 illustrates the minimum required delay for $O_{j,o}$ to process without interruption. Given this figure, $\frac{de_{j,o,t}}{p_{j,o'}}$ units of product related to $O_{j,o'}$ are produced during $de_{j,o,t}$. This amount is delivered to $O_{j,o}$ to produce $\frac{de_{j,o,t}}{\gamma_{j,o',o} \cdot p_{j,o'}}$ units of product during $\frac{de_{j,o,t} \cdot p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}}$ time unit. In this time $\frac{de_{j,o,t} \cdot p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}^2}$ units of product related to $O_{j,o'}$ produce again and deliver to $O_{j,o}$ to process. This series will continue until $O_{j,o}$ is complete.

If there is no interruption in operation $O_{j,o}$, its processing time is equal to:

$$\frac{de_{j,o,t} \cdot p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} + \frac{de_{j,o,t} \cdot (p_{j,o})^2}{(\gamma_{j,o',o})^2 \cdot (p_{j,o'})^2} + \dots + \frac{de_{j,o,t} \cdot (p_{j,o})^n}{(\gamma_{j,o',o})^n \cdot (p_{j,o'})^n} = de_{j,o,t} \cdot \sum_{k=1}^n \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)^k =$$

$$de_{j,o,t} \cdot \left(\frac{\left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right) - \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)^{n+1}}{1 - \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)} \right)$$

which defines a geometric series for $\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} < 1$

We can obtain the processing time of $O_{j,o}$ based on $de_{j,o,t}$ by tending n to infinity as follows.

$$\underbrace{x_{j,o,t} \cdot p_{j,o}}_{\text{process time}} = de_{j,o,t} \cdot \lim_{n \rightarrow \infty} \left(\frac{\left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right) - \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)^{n+1}}{1 - \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)} \right) = de_{j,o,t} \cdot \left(\frac{\left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)}{1 - \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'}} \right)} \right) = de_{j,o,t} \cdot \left(\frac{p_{j,o}}{\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}} \right)$$

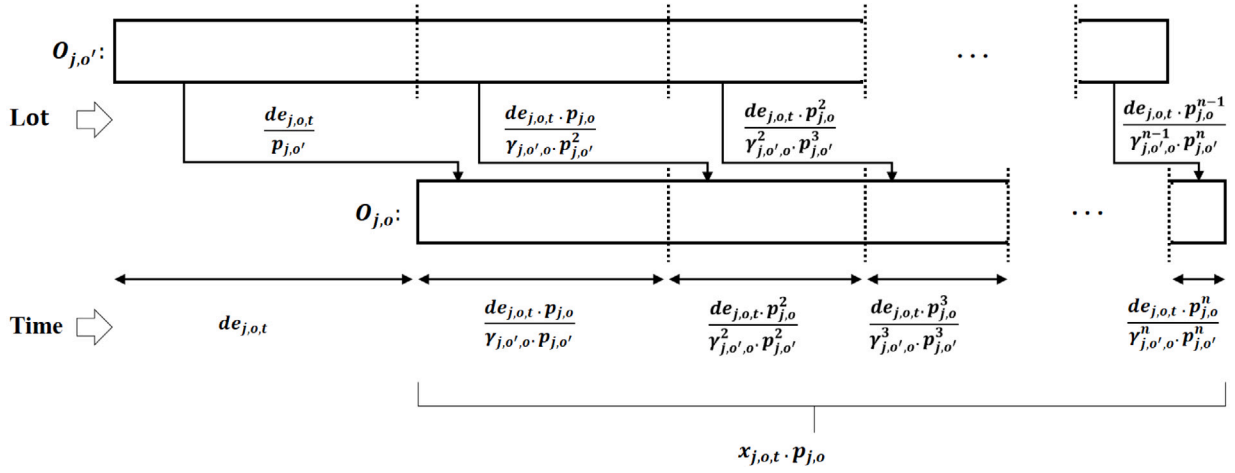


Fig. 2. Minimum delay of the successor operation for a process without interruption (continuous mode).

Therefore $de_{j,o,t}$, the minimum required delay in continuous mode in period t , and $f'_{j,o,t}$, the finish time of $O_{j,o}$ in the continuous mode in period t are equal to (15) and (16), respectively.

$$de_{j,o,t} = (\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot x_{j,o,t} \quad (15)$$

$$f'_{j,o,t} = \underbrace{(\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot x_{j,o,t}}_{\text{minimum required delay in continuous mode in period } t} + \underbrace{p_{j,o} \cdot x_{j,o,t}}_{\text{process time}} = p_{j,o'} \cdot \gamma_{j,o',o} \cdot x_{j,o,t} \quad (16)$$

To obtain the minimum required delay in the discrete mode (the product unit is discrete); first, we need to obtain $f_{j,o,t}$ as the minimum possible finish time of $O_{j,o}$ in the discrete mode to process without interruption in period t . Obviously, this time must be $p_{j,o}$ units greater than the time that the $(\gamma_{j,o',o} \cdot x_{j,o,t})$ th product of $O_{j,o'}$ is completed. Therefore, the minimum required delay in the discrete mode can be obtained as follows:

$$f_{j,o,t} = p_{j,o'} \cdot \gamma_{j,o',o} \cdot x_{j,o,t} + p_{j,o} = f'_{j,o,t} + p_{j,o}$$

$$= \underbrace{(\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot x_{j,o,t} + p_{j,o}}_{\text{minimum required delay in discrete mode in period } t} + \underbrace{p_{j,o} \cdot x_{j,o,t}}_{\text{process time}}$$

We have proven that even without considering the inventory for $O_{j,o'}$ at the end of period $t-1$, our overlap allows uninterrupted production. When considering inventory, the overlap is analogous, but instead of lot-size $x_{j,o,t}$ we should consider $(x_{j,o,t} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor)$ for obtaining the minimum required delay to start the last $(x_{j,o,t} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor)$ units of $O_{j,o}$. This time can be shifted $(\lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor \cdot p_{j,o})$ units to the left to obtain the minimum possible time to start $O_{j,o}$. Finally, the minimum required delay to start $O_{j,o}$ is as seen below, thus the proof is completed.

$$(\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot (x_{j,o,t} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor) + p_{j,o} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor \cdot p_{j,o}$$

$$= (\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot (x_{j,o,t} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor) + (1 - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor) \cdot p_{j,o}$$

$$(\gamma_{j,o',o} \cdot p_{j,o'} - p_{j,o}) \cdot (x_{j,o,t} - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor) + (1 - \lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor) \cdot p_{j,o} \quad \square$$

For the linearization of the model, an integer variable $u_{j,o',t}$ can replace $\lfloor \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} \rfloor$, while $u_{j,o',t} \geq \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}} - 1$ and $u_{j,o',t} \leq \frac{I_{j,o',o}(t-1)}{\gamma_{j,o',o}}$. For the

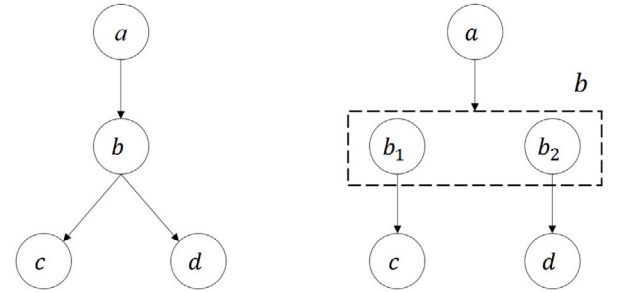


Fig. 3. Predecessor breakdown into the number of its successor branches.

better readability of the paper, we have considered the Gozinto factors equal to one ($\gamma_{j,o',o} = 1 \forall O_{j,o'}$). Therefore, the necessity of linearization is obviated, and $\gamma_{j,o',o}$ is omitted in the rest of the paper.

It should be noted that the Constraints (11) and (12) are valid for, at most one setup per operation in a period. Due to the triangle inequality of the setup time, there is, at most, one setup for each operation per period in the optimal solution of the proposed model. More information, including its proof, is provided in Appendix A.

Moreover, the minimum required delay between the start of two consecutive operations (according to Eq. (14)) does not force the manufacturer to transfer the material unit by unit from one level to the next. In other words, according to the optimal schedule, the decision maker can determine the size of the material transfer batches so that the minimum number of transfers between the two levels is performed.

4.1. Divergent structures

As mentioned previously, applying the proposed models for the divergent bill of materials requires some modifications. For this purpose, each predecessor operation must be duplicated in the number of its successor branches, as shown in Fig. 3.

For the structure shown in Fig. 3, Constraints (17) to (21) will be added to the original model. The notations are the same as in the original model, but the operation names have been used for the sake of simplicity instead of the (j, o) combinations.

$$x_{b,t} = x_{b_1,t} + x_{b_2,t} \quad (17)$$

$$I_{b,t} = I_{b_1,t} + I_{b_2,t} \quad (18)$$

$$y_{b_1,b_2,t} + y_{b_2,b_1,t} = z_{b,t} \quad (19)$$

$$s_{i,t} \geq s_{b,t} \quad \forall t, i \in [b_1, b_2] \quad (20)$$

$$s_{i',t} \geq s_{i,t} + x_{i,t} \cdot p_b - (1 - y_{i,i',t}) \cdot G \quad \forall t, i \in [b_1, b_2], i' \in [b_1, b_2] \mid i \neq i' \quad (21)$$

For the successor operations c and d (generally the successor operations with shared predecessor), Constraints (22) to (24) replace their corresponding Constraints (6), (11) and (12) in the original model respectively.

$$I_{i',t-1} - x_{i,t} + w_{i,t} \cdot G \geq 0 \quad \forall t, (i, i') \in [(c, b_1), (d, b_2)] \quad (22)$$

$$s_{i,t} + (1 - w_{i,t}) \cdot G \geq s_{i',t} + \gamma_{i'} \cdot p_{i'} - I_{i',t-1} \cdot p_i \quad \forall t, (i, i') \in [(c, b_1), (d, b_2)] \mid p_{i'} \leq p_i \quad (23)$$

$$s_{i,t} + (1 - w_{i,t}) \cdot G \geq s_{i',t} + (\gamma_{i'} \cdot p_{i'} - p_i) \cdot (x_{i,t} - I_{i',t-1}) + (1 - I_{i',t-1}) \cdot p_i \quad (24)$$

$$\forall t, (i, i') \in [(c, b_1), (d, b_2)] \mid p_{i'} > p_i$$

5. Solution approaches

The integrated lot-sizing and scheduling problem is computationally challenging. This problem is $\mathcal{NP}$ -hard when the classical single-item lot-sizing problem is extended by capacity constraint (Florian et al., 1980). The complexity of the problem increases drastically by transforming a single-item to a multi-item (Brügemann and Jahnke, 2000), and a single-level to a multi-level (Maes et al., 1991). Hence, many studies have focused on different ways to tackle this complexity.

This section develops two solution approaches for solving the proposed model. First, an iterative solution approach based on different variants of the Fix-and-Relax algorithm is developed and strengthened by a Fix-and-Optimize heuristic. This method is called FR-FO. Second, a powerful decomposition heuristic, called DH, is developed by combining mixed-integer programming (MIP) and constraint programming (CP) optimizers for the Master and sub-problems, respectively. The details of the proposed solution approaches are described below.

5.1. Fix-and-relax with fix-and-optimize

5.1.1. Fix-and-relax

The Fix-and-Relax (FR) heuristic is well-suited for different categories of problems, such as production planning, lot-sizing, scheduling, inventory planning, and supply chain management, which attains near optimum results within a reasonable computational time (Rodopl et al., 2020). This heuristic is a constrictive heuristic founded on decomposing the global problem into smaller sub-problems with fewer binary or integer variables. The sub-problems are solved sequentially using the information provided by the previous sub-problem until a complete solution is formed. In each sub-problem (SB), given to the complete set of binary or integer variables ($whole_window$), only a relatively small set of the binary variables are explicitly included, and the rest of them are fixed or relaxed. This small set of binary or integer variables is called ($window$) with a predefined size ($window_size$). The $window$ is updated according to a transition rule ($transition_rule$) while a rate of the binary or integer variables ($0 \leq overlap_rate \leq 1$) belonging to a previous $window$ are selected to be re-optimized in the new $window$, and the rest are fixed and added to a set of fixed variables ($fixed_window$). In each sub-problem, the variables not in the sets $window$ and $fixed_window$ are relaxed ($relaxed_window$) and allowed to take any real value within its particular domain. Algorithm 1 summarizes the pseudo-code of the FR heuristics proposed in this paper while the execution time of each sub-problem is restricted by value (max_time). As specified in line 3 of Algorithm 1, FR terminates when either all the variables in $whole_window$ are transferred to the $fixed_window$ or the execution time exceeds max_time . In this algorithm, s_{mip} , s_{lp} , and s_{fx}^* are the sets of variables belonging to

$window$, $relaxed_window$, and $fixed_window$, respectively. In each iteration of the algorithm, they are updated according to lines 5, 6, and 12, which collectively define the whole set of variables belonging to the sub-problem in the current iteration (SB). By optimizing SB , the new_window will be set according to line 9, and then after updating the $relaxed_window$, $fixed_window$, and $window$ (lines 4, 10, and 11), the algorithm continues until the termination conditions are met.

Algorithm 1: Pseudo-code of the proposed FR heuristics

```

input :  $whole\_window$ ,  $window\_size$ ,  $overlap\_rate$ ,
         $transition\_rule$ ,  $max\_time$ 
1  $fixed\_window \leftarrow \emptyset, S_{fx}^* \leftarrow \emptyset$ 
2  $window \leftarrow$ 
   INITIAL_WIN( $whole\_window, window\_size, transition\_rule$ )
3 while  $fixed\_window \neq whole\_window$  and
    $elapsed\ time \leq max\_time$  do
4    $relaxed\_window \leftarrow whole\_window \setminus window \setminus fixed\_window$ 
5    $S_{mip} \leftarrow \{x \mid x \in window\}$ 
6    $S_{lp} \leftarrow \{x \mid x \in relaxed\_window\}$ 
7    $SB \leftarrow (S_{fx}^* \cup S_{mip} \cup S_{lp})$ 
8    $S_{mip}^*, S_{lp}^* \leftarrow \text{Optimize}(SB)$ 
9    $new\_window \leftarrow$ 
      NEW_WIN( $whole\_window, window, overlap\_rate, transition\_rule$ )
10   $fixed\_window \leftarrow fixed\_window \cup (window \setminus new\_window)$ 
11   $window \leftarrow new\_window$ 
12   $S_{fx}^* \leftarrow S_{fx}^* \cup \{x^* \in S_{mip}^* \mid x \notin window\}$ 
13 return  $S_{fx}^*$ 

```

(i) How to select the binary variables included in the $window$ and (ii) how to update the $window$ along the $whole_window$ are two rules defined in the $transition_rule$. Three transition rules motivated by Toledo et al. (2015) are developed: (i) row-window, which moves in the row direction; (ii) column-window, which moves in the column direction; and (iii) value-window, where the variables belonging to a specific job and period with the closest average to 0.5 are selected and added to the $window$. After each iteration, the $window$ moves to the $step$ size ($step = |window_size \cdot overlap_rate|$). ($window_size - step$) variables are removed from the $window$ and are fixed in the next iteration, while $step$ new variables from $relaxed_window$ are added to the $window$ and enforced to be binary. The algorithm is repeated in this fashion until $fixed_window = whole_window$ or the time restriction is reached.

Fig. 4 illustrates three examples with 4 jobs and 6 periods, while 3 different transition rules are shown separately. P_t denotes period t , and VJ_j denotes the set of all the binary variables related to job j . Therefore, the pair (VJ_j, P_t) represents all the binary variables of job j in period t . Fig. 4(a) shows the row-window following the row direction where, in the first iteration, the variables (VJ_1, P_1) to (VJ_1, P_5) are enforced to be binary as the members of the $window$. After solving the first sub-problem the variables (VJ_1, P_1) and (VJ_1, P_2) are fixed to their optimal values and leave the $window$, the variables (VJ_1, P_6) and (VJ_2, P_1) are added to the $window$ and variables (VJ_1, P_3) to (VJ_1, P_5) are re-optimized in the next iteration. Fig. 4(b) shows the column-window following the column direction where a similar procedure is applied. In the value-window transition rule, an LP model is first solved with all the binary variables relaxed (between 0 and 1). Then the first $window_size$ variables with values closest to 0.5 are added to the $window$. In this transition rule, we assume the relaxed binary variables whose optimal values are close to one or zero have a lower priority to be included in the $window$ set in the initial iterations of the algorithm. Instead, we prefer variables with more indeterminate conditions (with an average closest to 0.5) for inclusion in the $window$ set. For example, in Fig. 4(c), it is supposed the arrays (VJ_4, P_1) , (VJ_1, P_2) , (VJ_3, P_2) , (VJ_2, P_4) , and (VJ_4, P_5) have the closest average to 0.5. Therefore, they are selected and added to the $window$ to be optimized in the binary mode. The arrays related to the first columns are preferred for selection

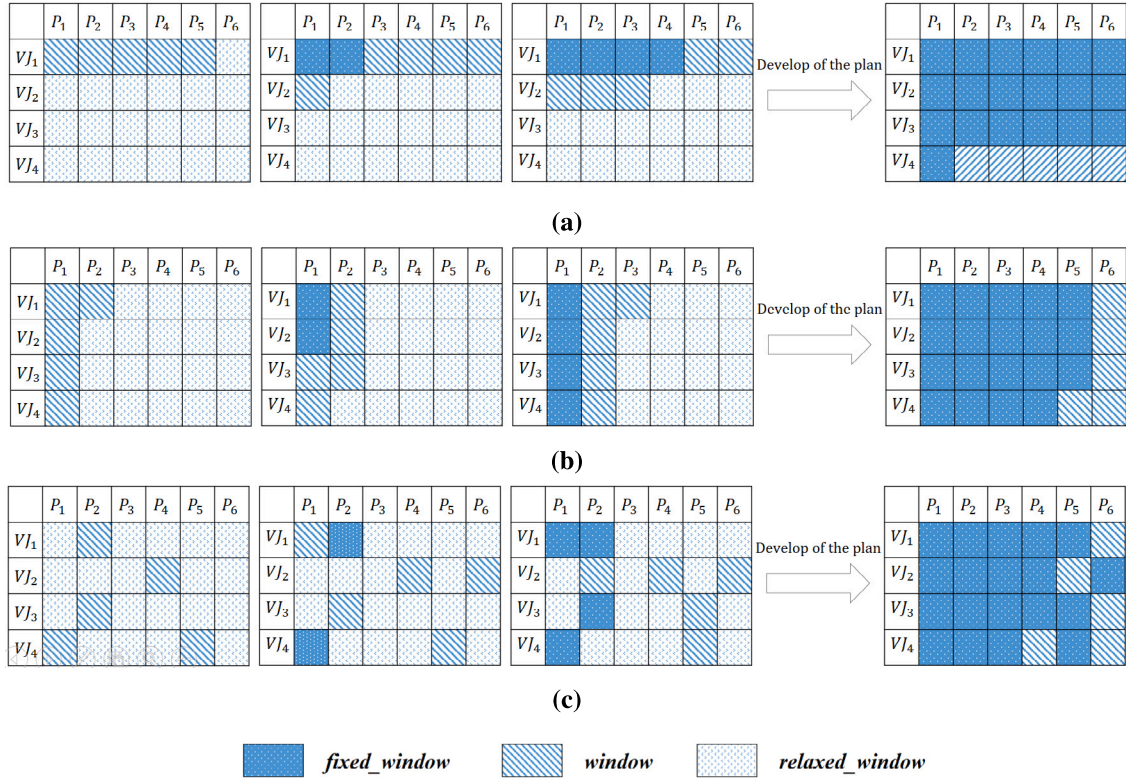


Fig. 4. Illustration of the proposed FR heuristics in an iterative pattern.

between the arrays with the same average of variables. If they belong to the same column, the arrays related to the first rows will be selected. The arrays related to the first columns and rows also prioritize leaving the *window* and adding to the *fixed_window*. For example, in the first matrix of Fig. 4(c), the variables belonging to (VJ_4, P_1) and (VJ_1, P_2) are fixed and removed from the *window*.

5.1.2. Fix-and-optimize

A Fix-and-Optimize (FO) heuristic is developed to improve the solutions built by the proposed FR heuristics. The reasoning behind FO is that the binary setup $(z_{j,o,t})$ and sequence variables $(w_{j,o,t})$ and $(y_{j,o,j',o',t})$ impose most of the numerical effort in the solution approach. Therefore, FO selects and solves a series of them iteratively through a systematic manner that can be solved quickly (Sahling et al., 2009). In each iteration, a sub-problem is solved while most of the binary variables in the sub-problem are set to a fixed value obtained by the FR heuristics. In the meantime, a smaller part of the binary variables and all the continuous variables are obtained optimally. Hence, the FO heuristic leads to an improvement in the objective function through:

- Moving the whole or part of the production of an item from one period to another period.
- Changing the sequence of items within a particular period.

In the FO heuristic, the performance of the algorithm and the quality of the final solution are directly affected by the way of choosing the variables that need to be fixed, as well as the number of them (Sahling et al., 2009). Therefore, a rolling window is also defined and executed for the proposed FO. The execution process is similar to the proposed FR heuristics. In each iteration, a *window* of binary variables is re-optimized within a limited time (*max_time*) while all the variables outside the *window* are kept fixed. Then, the *window* is moved forward according to the *transition_rule* so that a subset of the binary variables (*overlap_rate*) belonging to the previous *window* transfer to the new

window. Four transition rules are proposed to move the *window*. Similar to the proposed FR heuristics, the (i) row-window, which moves in the rows direction, and (ii) column-window, which does the same along with the columns, are also defined in the proposed FO (see Figs. 4(a) and 4(b)). The (iii) rectangle-window following the row and (iv) column directions are two new transition rules applied in the proposed FO. The square-window moves along both row and column directions until all the binary variables are re-optimized at least once.

Fig. 5 illustrates the execution of the FO with the rectangle-window of size 3×2 following the row (Fig. 5(a)) and column (Fig. 5(b)) directions. The *overlap_rate* is assumed to be 50%, and the *whole_window* contains all the binary variables belonging to 4 jobs and 5 periods.

Determining the *window_size* is an important part of the method so that the maximum improvement can be obtained in a restricted time if this factor is appropriately selected. Increasing the *window_size* may culminate in higher-quality solutions due to the increased search space. Nonetheless, this procedure cannot be efficient if a large number of binary variables are selected and assigned to the *window*. All the parameters of the proposed FR heuristics and the FO are determined experimentally.

5.2. Decomposition heuristic

Separating a large model into several smaller sub-problems and solving them in an iterative process instead of solving the original model is the main idea of the decomposition strategy. This strategy usually works well in tackling the complexity of large-sized problems. Logic-based Benders decomposition (LBBD) is a well-known method and an extension of the classical Benders algorithm. This method decomposes a model into sub-models called the master problem (MP) and sub-problem (SP). In contrast to the classical Benders decomposition algorithm, the SP in the LBBD does not have to be a continuous linear program (Barzanji et al., 2020). Accordingly, by considering the binary variables in both the lot-sizing and scheduling sub-problems, we are

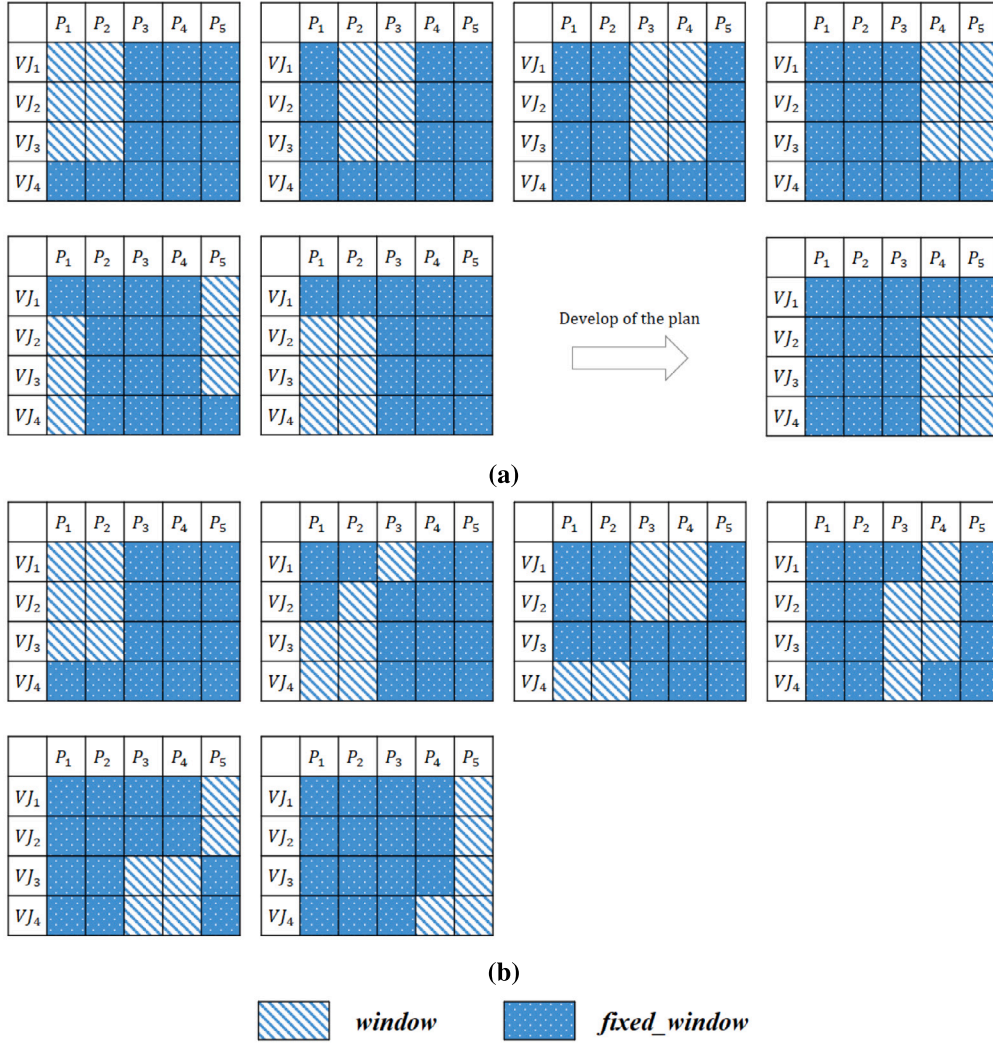


Fig. 5. Illustration of the proposed FO in an iterative pattern.

motivated to develop a novel decomposition heuristic (DH) inspired by the logic-based Benders decomposition algorithm. In the proposed decomposition heuristic, MP and SP are formulated using mixed integer programming (MIP) and constraint programming (CP), respectively. This heuristic solves the MP, to obtain an optimal objective value, regardless of the detailed schedule (sequencing sub-problem). Hence, the SP may be infeasible considering the lot-size values from MP. Due to the MIP nature of SP in the LBBD algorithm, the classical optimality and feasibility cuts extracted from the dual linear sub-problem are not applied in this algorithm. Instead, these cuts are generated based on the logical relations and rules of the problem (Hooker, 2007). In our problem, to satisfy the feasibility of the SP, a customized feasibility cut is proposed and added to the MP, which is solved to obtain updated lot sizes as the input parameters for the next SP. The algorithm terminates when all the SPs obtain a feasible solution during the planning horizon. The general procedure of the proposed decomposition heuristic is depicted in Fig. 6.

5.2.1. Master problem

In the MP, a relaxed version of the original model is solved, which is computationally straightforward. The decision variables related to the lot-sizing problem consist of the assignment of the operations to the periods ($z_{j,o,t}$), lot-size ($x_{j,o,t}$), and operations inventory ($I_{j,o,t}$) that are obtained in the proposed MP. The initial form of the proposed MP

is defined below, while the objective value of the MP makes a lower bound for the original problem.

Min TSC

Subject to:

Constraint sets (2), (3), (4), and (5).

$$\{x_{j,o,t}, I_{j,o,t}\} \in \mathbb{R}^+; \{z_{j,o,t}\} \in \{0, 1\} \quad \forall t, \forall(j, o) \quad (25)$$

5.2.2. Sub-problems

In the proposed DH, the SP decomposes into several particular scheduling problems (SPs) separately for each period. Each scheduling problem investigates whether there is a feasible schedule taking the lot-size of the operations into account following the optimum solution of the previous MP. If not, a feasibility cut is generated, as described in the next section.

The time window size (L) is relaxed as identifying the longest path (critical path) in the scheduling network, which is essential for generating the feasibility cut. This relaxation always provides a feasible solution for the SPs, but is infeasible for the original model when the critical path length is greater than the length of period L . To obtain the size of the longest path, the variable C_{max_t} is introduced as the maximum completion time of the operations (makespan) in period t . This variable is minimized as the objective function of the SPs (26).

$$\text{Min } C_{max_t} \quad (26)$$

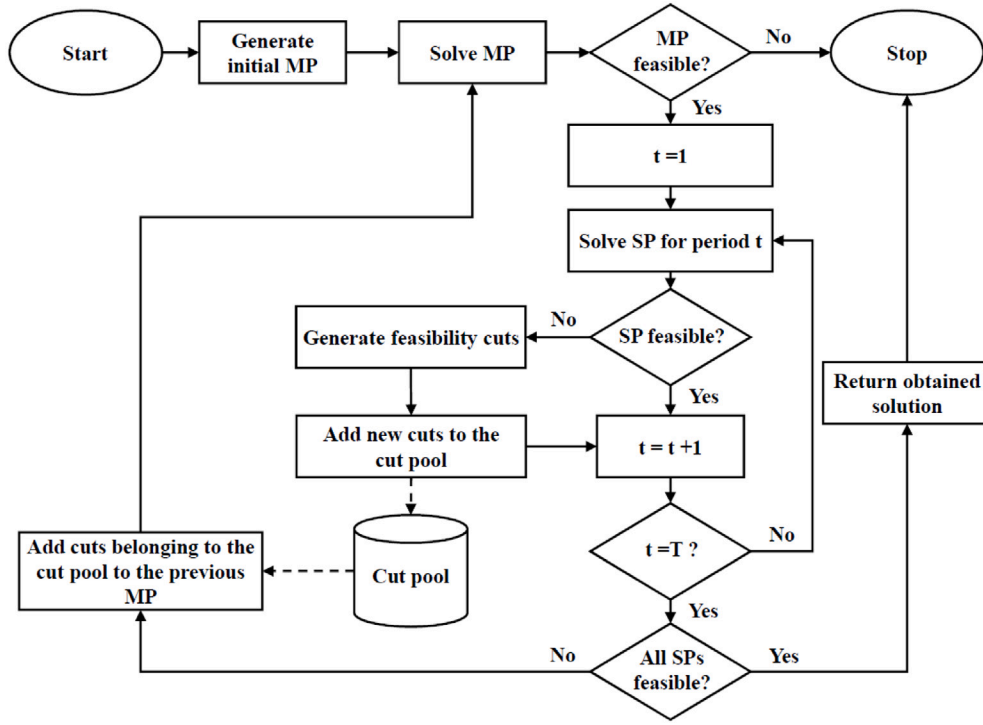


Fig. 6. The flow chart of the proposed decomposition heuristic.

Forasmuch as the SPs are just a feasibility model, Constraint (27) ensures the SP stops after finding the first feasible solution for period t of the original model.

$$C_{max_t} \geq L \quad (27)$$

The SPs are formulated in the form of constraint programming (CP) due to the advantages of the CP for solving the proposed SPs compared to the MIP model. In our case, the CP formulation is able to solve the SPs 10 times faster than the MIP on average. In the SPs, the variables $x_{j,o,t}$, $I_{j,o,t}$, $z_{j,o,t}$, and $w_{j,o,t}$ are respectively fixed to the values $\bar{x}_{j,o,t}$, $\bar{I}_{j,o,t}$, $\bar{z}_{j,o,t}$, and $\bar{w}_{j,o,t}$ that they receive in the solution of the previous MP.

Concerning the CP formulation, first, an *optional interval variable* ($T_{j,o}$) is introduced for each operation $O_{j,o}$. An *optional interval variable* can be set to $\{present\}$ or $\{absent\}$. This variable is removed from the schedule if its value is set to $\{absent\}$. $T_{j,o}$ is set to $\{present\}$ if $\bar{z}_{j,o,t} = 1$ and $\{absent\}$ otherwise. The predicate *Presence_of* is used to determine the presence or absence of the interval variable in the schedule. For this end, Constraint (28) is added to the SPs.

$$Presence_of(T_{j,o}) = \bar{z}_{j,o,t} \quad \forall(j, o) \quad (28)$$

The predicates *Length_of* and *Start_of* for $T_{j,o}$ are as follows. Similar to Constraint (9), Constraint (30) is always a valid inequality for the SPs.

$$Length_of(T_{j,o}) = \bar{x}_{j,o,t} \cdot p_{j,o} \quad \forall(j, o) \quad (29)$$

$$Start_of(T_{j,o}) \geq \delta_{0,j,o} \cdot \bar{z}_{j,o,t} \quad \forall(j, o) \quad (30)$$

Then a *sequence variable* (Seq_m) is defined in accordance to (31). This variable consists of all $T_{j,o}$ that $a_{j,o} = m$.

$$Seq_m = Sequence_var\left(\bigcup_{\substack{\forall(j,o) \\ a_{j,o}=m}} T_{j,o}\right) \quad \forall m \quad (31)$$

Constraint (32) enforces that the interval variable $T_{j,o}$ should start at least $Lag_{j,o}$ time units later than interval variable $T_{j,o'}$ if $\bar{w}_{j,o,t} = 1$ and

$$O_{j,o'} \in Q_{j,o}.$$

$$Start_before_start(T_{j,o'}, T_{j,o}, Lag_{j,o}) \quad \forall(j, o, o') \mid O_{j,o'} \in Q_{j,o} \& \bar{w}_{j,o,t} = 1 \quad (32)$$

The value of $Lag_{j,o}$ can be positive or negative according to (33).

$$Lag_{j,o} = \begin{cases} p_{j,o'} & \text{if } p_{j,o'} \leq p_{j,o} \\ (p_{j,o'} - p_{j,o}) \cdot (\bar{x}_{j,o,t} - \bar{I}_{j,o',t(t-1)}) + (1 - \bar{I}_{j,o',t(t-1)}) \cdot p_{j,o} & \text{if } p_{j,o'} > p_{j,o} \end{cases} \quad (33)$$

Constraint (34) uses the *No_overlap* function for the interval variables which share the same machine. This constraint ensures a sequence of interval variables so that any interval variable needs to be completed before the start of its next interval variable in the sequence. The S_m^I is the matrix $(|Seq_m| \times |Seq_m|)$ while component (i, k) of the matrix is equal to $\delta_{j,o,j',o'}$ if $T_{j,o}$ and $T_{j',o'}$ are the i th and k th elements of Seq_m respectively.

$$No_overlap(Seq_m, S_m^I) \quad \forall m \quad (34)$$

The variable C_{max_t} is computed by adding Constraint (35) in the SP.

$$C_{max_t} \geq End_of(T_{j,o}) \quad \forall(j, o) \quad (35)$$

5.2.3. Feasibility cut

The convergence of the logic-based Benders approach significantly depends on the feasibility or optimality cuts. As mentioned before, in the proposed DH, the feasibility cuts must be generated based on some logical relations that exist in the problem. Hence, the critical issue here is recognizing and defining strong cuts; otherwise, slow convergence or cycling is possible. In our case, after solving each SP model, if the feasible solution is not found ($C_{max_t} > L$), an effective feasibility cut is generated and added to a cut pool. At the end of each iteration, all the collected feasibility cuts in the cut pool are added to the previous

MP. These cuts remove a part of MP's solution space and avoid the last obtained infeasible solution in the next iterations.

The developed feasibility cut is formed based on simple logic: the length of the longest path in the scheduling network (critical path) must be less than or equal to the length of the period. It should be noted that the proposed feasibility cut may lead to the elimination of optimal solutions. Although this cut does not guarantee the optimality of the final solutions, the experimental results prove the high quality of the final solutions. To create the feasibility cut, the following four steps must be performed.

(a) Finding the critical paths in the scheduling network:

A simple algorithm is applied to find the critical paths. First, the scheduling network is created after identifying the dependencies of the operations. The scheduling network is a mixed graph in which nodes represent tasks and timing constraints are shown by directed edges. The edges represent two types of timing constraints as follows:

- timing constraints due to establishing the material balance between two consecutive operations of the same job. A directed edge connects node $O_{j,o'}$ to $O_{j,o}$; if $O_{j,o'} \in Q_{j,o}$.
- timing constraints when two operations $O_{j,o}$ and $O_{j',o'}$ perform on a same machine. A directed edge connects node $O_{j,o}$ to $O_{j',o'}$ if $a_{j,o} = a_{j',o'}$; and $O_{j',o'}$ must be performed immediately after $O_{j,o}$.

Then the longest path in the scheduling network, which we call the "critical path" is identified. First, the critical operations are determined by estimating their earliest finish time (*EFT*) and latest finish time (*LFT*). The operations are critical if $LFT - EFT = 0$ for them. Moreover, a path is critical if all the operations belonging to it are critical.

(b) Finding the types of operations belonging to the critical path:

To generate the feasibility cut, we need to know how much of the critical path's length is occupied by each operation belonging to the path (the share of each operation should be specified). The share of each operation in the critical path is a function that depends on its before and after operations in the path. To this end, all the operations ($O_{j,o}$) belonging to the critical path are categorized into four types ($type_{j,o}$) concerning their before and after operations in the path. (i) Using a shared resource and (ii) creating the material balance are two reasons that made the dependencies between the operations in the path. Hence, the $type_{j,o}$ is defined as follows:

$$type_{j,o} = \begin{cases} RR & \text{Operation before: Shared Resource;} \\ & \text{Operation after: Shared Resource} \\ MM & \text{Operation before: Material balance;} \\ & \text{Operation after: Material balance} \\ RM & \text{Operation before: Shared Resource;} \\ & \text{Operation after: Material balance} \\ MR & \text{Operation before: Material balance;} \\ & \text{Operation after: Shared Resource} \end{cases}$$

The first operation in the path can be categorized in the $type$ RR or RM depending on its operation after. Likewise, the last operation on the path can be classified in the $type$ RR or MR depending on its operation beforehand.

(c) Finding the equations representing the share of each operation from the length of the critical path:

Each operation's share of the critical path is mathematically determined in this stage. The share for operation $O_{j,o}$ in period

t is in accordance with (36):

$$\begin{cases} x_{j,o,t} \cdot p_{j,o} + \delta_{j',o',j,o} & \text{if } O_{j,o} \text{ is in type RR; and } e_{j,o} = O_{j',o'} \\ Del_{j,o,t} & \text{if } O_{j,o} \text{ is in type MM} \\ Del_{j,o,t} + \delta_{j',o',j,o} & \text{if } O_{j,o} \text{ is in type RM; and } e_{j,o} = O_{j',o'} \\ (x_{j,o,t} - I_{j,o',t-1}) \cdot p_{j,o} & \text{if } O_{j,o} \text{ is in type MR; and } e_{j,o} = O_{j,o'} \mid O_{j,o'} \in Q_{j,o} \end{cases} \quad (36)$$

In (36), $e_{j,o}$ is the operation directly before $O_{j,o}$ in the path. Moreover, the value of $Del_{j,o,t}$ is equal to $p_{j,o}$ if $p_{j,o} \leq p_{j,o'}$; otherwise $(p_{j,o} - p_{j,o'}) \cdot (x_{j,o',t} - I_{j,o',t-1}) + (1 - I_{j,o',t-1}) \cdot p_{j,o'} \mid O_{j,o'} \in Q_{j,o'}$. The equation representing the critical path length is equal to the sum of the shares related to each operation belonging to the path.

(d) Generate the feasibility cuts:

Finally, the feasibility cuts are established by an inequality that ensures the summation of the share of operations according to equations in step (c), must be smaller than or equal to the length of the period. (L). The feasibility cut is in accordance with (37). ρ is the set of operations belonging to the critical path, and $\delta_{0,j'',o''}$ is the setup time of the operation $O_{j'',o''}$ while $O_{j'',o''}$ is the first operation executed in the critical path.

$$\begin{aligned} & \sum_{\substack{\forall (j,o) \in \rho \\ type_{j,o} = RR}} x_{j,o,t} \cdot p_{j,o} + \sum_{\substack{\forall (j,o) \in \rho \\ type_{j,o} \in \{MM, RM\}}} Del_{j,o,t} \\ & + \sum_{\substack{\forall (j,o) \in \rho \\ type_{j,o} = MR \\ e_{j,o} = O_{j,o'}}} (x_{j,o,t} - I_{j,o',t-1}) \cdot p_{j,o} \\ & + \delta_{0,j'',o''} + \sum_{\substack{\forall (j,o) \in \rho \\ type_{j,o} \in \{RR, RM\} \\ e_{j,o} = O_{j',o'}}} \delta_{j',o',j,o} \leq L \end{aligned} \quad (37)$$

6. Computational experiments

This section presents the evaluation results of the proposed model, algorithms, and their improvements. The MIP and the CP models are coded and solved by CPLEX and IBM CP optimizer accessed via Python, respectively. All the models run on a PC with a Core i7 processor and 8 GB RAM. Our numerical experiments used two data groups, each containing 20 random instances of different sizes. In the first data sets (labeled $\{\mathcal{T}\}$), the length of the periods and the capacity of the machines are considered as tight as possible. In the second group (labeled $\{\mathcal{W}\}$), the length of the periods and the capacity of the machines are kept wider to obtain easier instances with a lower degree of complexity. All the instances are labeled $(\{\mathcal{T}\}/(\{\mathcal{W}\})\text{J-O-M-T})$, which indicates the group of data ($\{\mathcal{T}\}/(\{\mathcal{W}\})$), the number of jobs (J), the number of operations per job (O), the number of machines (M), and the number of periods (T), respectively. In the generated data, the capacity of the machines in the $\{\mathcal{T}\}$ data set is on average 10% less than in the $\{\mathcal{W}\}$ data set, and the length of the periods in the $\{\mathcal{T}\}$ data set is also on average 20% less than in the $\{\mathcal{W}\}$ data set. A Relative Percentage Deviation (RPD) criterion is used to evaluate the proposed model and solution methods and demonstrate their effectiveness. The RPD for solution method A is calculated based on the deviation of its final solution to the best solution achieved by all the solution methods according to (38).

$$RPD = \frac{obj_A - \min_i \{obj_i\}}{\min_i \{obj_i\}} \quad (38)$$

Six key performance indicators (KPIs) are investigated to evaluate the performance of the proposed model, namely (i) feasibility, (ii) solution quality, (iii) re-schedulability, (iv) responsiveness, (v) work-in-process, and (vi) machine idle time. These KPIs reflect the efficiency

of the proposed model compared to some well-known models in the literature, where the models are solved many times under a protocol in which the demands are randomly changed at each iteration.

6.1. Generating data sets

To the best of our knowledge, there are no standard datasets in the literature for capacitated lot-sizing and job shop scheduling to be considered as the benchmark. Most of the studies investigated the results on a randomly generated dataset (Ouenniche and Bector, 1998; Huang, 2010; Gomez Urrutia et al., 2014; Karimi-Nasab et al., 2015; Giglio et al., 2017). Hence, we generated the number of periods, number of machines, demands, setup costs, inventory holding costs, and production costs in accordance with (Giglio et al., 2017). Three different planning periods (5, 4, and 6), the different number of machines between [3-15], and the different number of jobs between [3-20] are considered to generate the instances. The different number of operations per job is also considered between [3-20]. The total number of operations ($J \cdot O$) ranges from 12 to 200, covering a wide range of instances from a small to large size. It is possible to process two or more operations belonging to one job on one machine. The lower and upper bounds for the processing time of the operations on the machines are assumed to be 0.15 and 0.5, respectively. The lower and upper bounds for the demand per period are assumed to be 100 and 300, respectively. The setup cost, production cost, and inventory holding costs have been generated according to the uniform distributions between [10, 30], [0.1, 0.3], and [2, 5], respectively. The sequence-dependent setup time has also been generated according to a uniform distribution between [3, 6].

The available capacity of the machines in each period is computed through a modified version of the equation proposed by Gomez Urrutia et al. (2014) as follows:

$$C_{m,t} = cap \cdot \sum_{j=1}^J \sum_{o=1}^{h_j} (p_{j,o} \cdot D_{j,t} + \max_{j',o'} \{ \delta_{j',o',j,o} | a_{j,o} = a_{j',o'} \}) \quad (39)$$

Where cap is the capacity ratio between 0.1 and 0.5 depending on instances, which is multiplied by the maximum required time to satisfy the total demand of period t using only one machine considering the nominal processing time of the operations. All randomly generated instances are available on: <https://shorturl.at/rBLMQ>.

6.2. Numerical results

In this section, we first investigate the efficiency of our proposed model vs. the well-known models in the literature, including the PLSP presented by Kimms (1999) (PLSP1), the PLSP presented by Stadtler (2011) (PLSP2), the GLSP proposed by Seeanner and Meyr (2013), and the continuous time lot-sizing and scheduling (CTLSP) (Ramezani et al., 2013c). All these models are presented in the supplementary material. The customized models are also compared with our proposed model regarding the number of variables and constraints. The models and their comparison results are available at: <https://shorturl.at/loD67>. In the second experiment, the performance of the proposed solution methods is investigated and compared in terms of the quality of the final solution and the computational time. Moreover, some statistical analyses are performed to confirm the accuracy of the results.

6.2.1. Analysis of the proposed models

The results of 20 generated instances, obtained by the proposed model and the models extracted from literature, are presented in Table 1. For this comparison, the instances of the second data group ($\{W\}$) with the wider length of periods and capacity of the machines have been used. The models cannot provide a feasible solution according to the specified time in most instances of the first data group ($\{T\}$). Therefore, this group of data is neglected for the performance

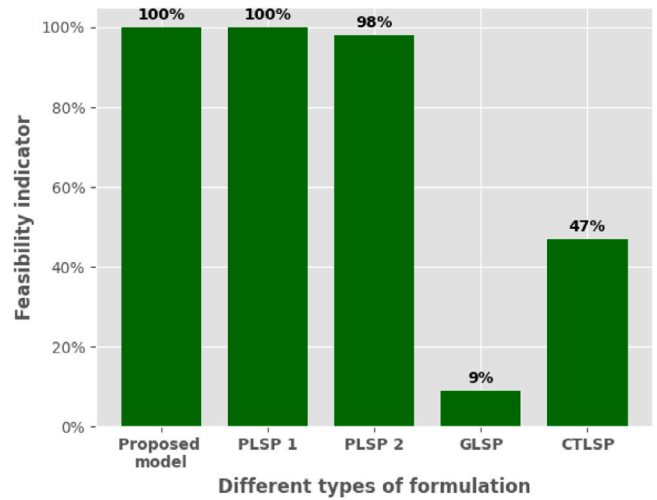


Fig. 7. The feasibility indicator for the given types of models.

analysis of the models. We set the time limit for solving an MIP model to 3600 s while the commercial MIP solvers solve the models. The RPD, CPU time, and the ability of the models to solve problems with different sizes can be compared. In this table, the CPU time column with a value greater than 3600 s indicates the instances where the model did not prove the solution's optimality or could not find a feasible solution within the time limit. As shown in Table 1, the proposed reformulation significantly impacts the results, especially regarding the computational time and the size of the problems. The small-bucket formulations cannot solve instances with more than 50 operations, while the proposed model solved instances that are twice as large. This improvement is more prominent when their CPU times are compared. The average CPU time for the instances solved by the proposed model is 967.5 s, while this time is 1593 s for the CTLSP and more than 3600 s for other models. Regarding the quality of the obtained results, there is an obvious improvement in the RPD criterion of the proposed reformulation. On average, 25%, 18%, 12%, and 8.5% of the RPD Gap is observed for PLSP 1, PLSP 2, GLSP, and CTLSP, respectively. The difference between the optimal solution of the proposed model and the CTLSP as another big-bucket time formulation is an important point that can be noticed in Table 1. This difference is more than 20% for some problems like 3-10-5-5. Therefore, ignoring the lot-streaming in CTLSP is not a cost-effective approach for the production processes where lot-streaming is possible.

6.2.1.1. Feasibility. The feasibility indicates the percentage of experiments that a feasible schedule can be found by a given model type during 600 s of computational time. An instance is un-schedulable when the system cannot meet the demands at their specified deadlines. Fig. 7 illustrates a comparison of the feasibility indicator in the different models. The feasibility indicator has been computed based on Monte Carlo simulation. During the simulation, 100 different randomly generated instances (with dimensions 3-10-5-5) are solved by the different types of models, considering a restriction of the computation time to 600 s. Finally, the percentage of the number of instances for which a given model has been able to find a feasible solution is presented as the feasibility indicator. Fig. 7 shows that the proposed model was able to find a feasible solution for all instances, with a feasibility indicator of 100%. While the small-bucket time formulations (PLSP 1 and PLSP 2) show desirable performance in terms of feasibility, GLSP and CTLSP fall significantly on this indicator. They failed to provide a feasible solution for a significant part of the instances.

Table 1

Comparison of the proposed model and the models extracted from the literature.

Total number of operations	Instances ((W)J-O-M-T)	Proposed model			PLSP 1			PLSP 2			GLSP			CTLSP		
		CPU time (s)	Objective function	RPD	CPU time (s)	Objective function	RPD	CPU time (s)	Objective function	RPD	CPU time (s)	Objective function	RPD	CPU time (s)	Objective function	RPD
12	{W}3-4-3-5	0.5	3982.6	0.00	>3600	3982.7	0.00	>3600	4007.8	0.01	>3600	3982.7	0.00	1.2	4071.5	0.02
12	{W}4-3-3-6	0.4	5108.1	0.00	>3600	5108.1	0.00	>3600	5153.5	0.01	>3600	5108.1	0.00	0.7	5223.8	0.02
30	{W}6-5-5-4	72.9	7967.7	0.00	>3600	10695.2	0.34	>3600	10379.7	0.30	>3600	10602.3	0.33	>3600	10682.7	0.34
30	{W}3-10-5-5	2.0	8807.5	0.00	>3600	12284.8	0.39	>3600	11141.1	0.26	>3600	10084.5	0.14	86.8	10779.5	0.22
30	{W}5-6-5-6	7.4	9601.4	0.00	>3600	13768.9	0.43	>3600	11823.0	0.23	>3600	10648.7	0.11	>3600	10788.8	0.12
50	{W}5-10-5-4	24.5	10766.9	0.00	>3600	13247.4	0.23	>3600	13193.5	0.23	>3600	–	–	>3600	11120.3	0.03
50	{W}5-10-7-5	316.1	12992.5	0.00	>3600	18658.7	0.44	>3600	17370.0	0.34	>3600	–	–	>3600	–	–
50	{W}10-5-5-6	40.9	15531.8	0.00	>3600	17948.3	0.16	>3600	16669.1	0.07	>3600	–	–	58.4	15531.8	0.00
80	{W}10-8-5-4	>3600	17336.1	0.00	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
80	{W}8-10-5-5	1394.1	21606.7	0.00	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
80	{W}20-4-4-6	462.2	25178.0	0.00	>3600	–	–	>3600	–	–	>3600	–	–	1067.7	25178.0	0.00
90	{W}6-15-10-5	>3600	22767.2	0.00	>3600	–	–	>3600	–	–	>3600	–	–	–	infeasible	–
100	{W}10-10-5-4	736.7	20581.4	0.00	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
100	{W}20-5-5-5	654.4	26476.1	0.00	>3600	–	–	>3600	–	–	>3600	–	–	2322.2	26476.1	0.00
100	{W}5-20-10-6	>3600	– ^a	–	>3600	–	–	>3600	–	–	>3600	–	–	–	infeasible	–
120	{W}10-12-8-5	>3600	34834.8	0.00	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
150	{W}10-15-10-4	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
160	{W}10-16-10-6	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
180	{W}12-15-10-5	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–
200	{W}10-20-15-4	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–	>3600	–	–

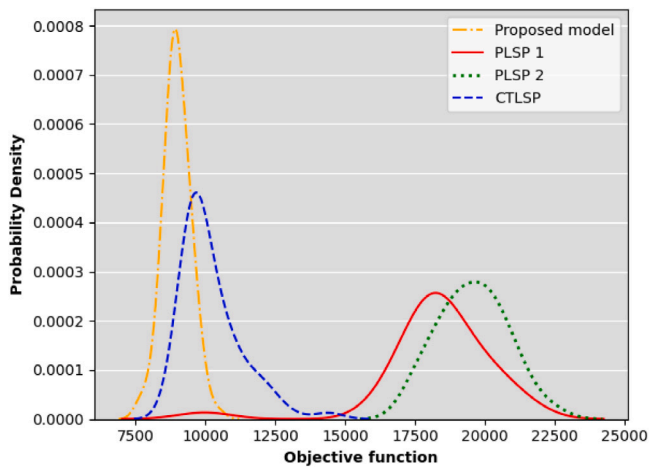
^aNo feasible solution found in the time limit.

Fig. 8. The solution quality indicator for the given types of models.

6.2.1.2. Solution quality. In the solution quality indicator, the objective function value is compared between certain types of models after 600 s of computational time. The KDE plot demonstrated in Fig. 8 is a probability density of the solution quality indicator that is smoothed by the Kernel smoothing technique (Wand and Jones, 1994). This figure shows the best obtained objective functions obtained by the proposed model, PLSP 1, PLSP 2, and CTLSP for 100 randomly generated instances with dimensions 3-10-5-5. The instances are generated following Section 6.1. In this figure, the proposed model with an average objective function equal to 8981.4 obtained better results than PLSP 1, PLSP 2, and CTLSP with an average objective function equal to 18313.1, 19564.6, and 10210.8, respectively. In this figure, all of the solutions obtained by the proposed model and most solutions obtained by CTLSP are optimal, while PLSP 1 and PLSP 2 could not reach the optimal solution in most iterations. These results also show that applying the lot-streaming strategy improves the operational costs by 7% on average compared to the case in which it is omitted (CTLSP).

6.2.1.3. Re-schedulability. In the manufacturing industries, there are many unexpected disruptions during the planning horizon, so that the original schedule may become invalid after a disturbance. Therefore, in many cases, rescheduling is essential in order to minimize the effects of this disruption and find a new schedule (Salido et al., 2017). In this section, we introduce the re-schedulability indicator that measures

the performance of the proposed model in providing a new schedule reacting to random disruptions in the demand and processing time during the planning horizon. To this end, the model is solved by applying a rolling planning approach in both i) periodic and ii) event trigger modes to find a new schedule. The periodic rolling planning approach is developed to deal with random changes in the demand. In this approach, it is assumed that some new demands with random amounts are added to the system at the end of each period. Therefore, the remaining operations are rescheduled at the beginning of a new period. The event trigger rolling planning approach is developed to deal with random disruptions in the processing time where the processing time of some randomly selected operations is subject to uncertainty. In the event trigger rolling planning, the rescheduling is performed as soon as the processing time of an operation changes, and the remaining operations are rescheduled immediately afterwards. Figs. 9 and 10 illustrate the re-schedulability indicator with a disruption in the demand and processing time, respectively. For measuring the re-schedulability indicator, we solved 3000 randomly generated instances with dimensions 3-10-5-5. The 3000 instances have been solved in 100 batches, including 30 instances. In the histograms of Figs. 9 and 10, the density of the number of instances out of 30 are shown in which a certain model was able to reschedule them during the whole planning horizon. These figures illustrate the effectiveness of the lot-streaming strategy in the re-schedulability indicator, where a comparison of the ILSP with lot-streaming (the proposed model) and without lot-streaming (CTLSP) is depicted. Fig. 9 shows that the proposed model with lot-streaming is significantly more effective in reducing the number of un-schedulable instances than the models without lot-streaming. Using the proposed model, the average number of rescheduled instances is increased from 14.4 to 24.7 out of 30 (40% improvement). Moreover, according to Fig. 10, the proposed model has a higher rescheduling capability against the processing time disruptions compared to the ILSP models without lot-streaming, while the result of this simulation also shows a 25% improvement in the re-schedulability indicator (the average of the rescheduled instances increased from 18.8 to 23.6 out of 30).

In addition, we measured the average value of the gap between the objective functions after and before rescheduling as another criterion for rescheduling performance. This experiment is performed with randomly generated instances with dimensions 3-10-5-5, and finally, their average gap is determined. In this experiment, at the end of the second period, we increased the demand for the upcoming periods equally in both the proposed model and CTLSP. The average gap between the objective functions after and before rescheduling in the proposed

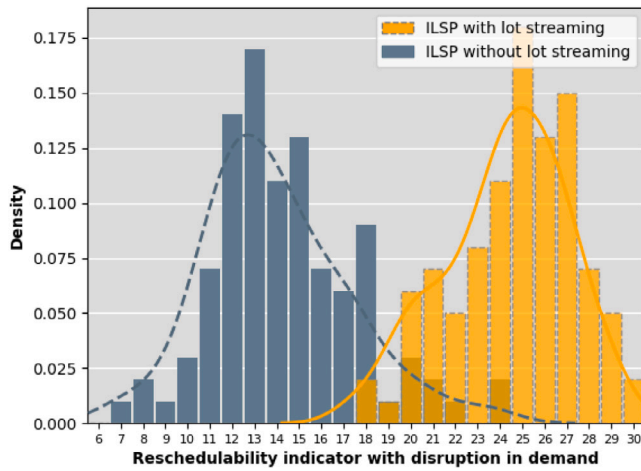


Fig. 9. The re-schedulability indicator with a disruption in the demand.

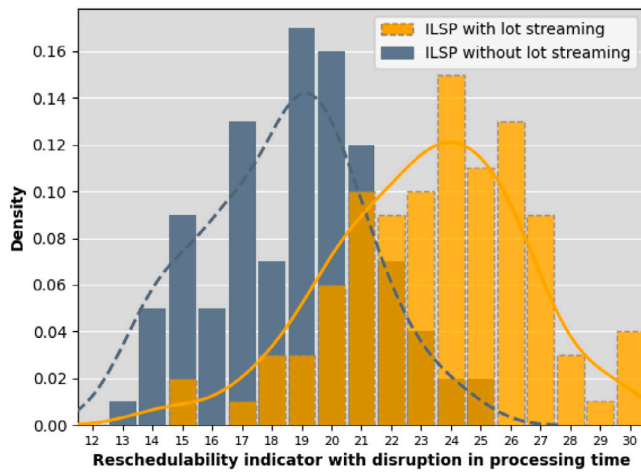


Fig. 10. The re-schedulability indicator with a disruption in the processing time.

model and CTLSP is 36% and 52%, respectively. The results of this experiment show that the proposed model has not only a higher ability to regenerate a feasible schedule but that its new schedules are also of higher quality.

6.2.1.4. Responsiveness. In general, the responsiveness can contain a set of time-dependent indicators representing the speed of responding to the demands according to their importance. Fig. 11 illustrates the KDE plot of the responsiveness indicator obtained by the proposed model with lot-streaming and without lot-streaming, while the models solved different instances with dimensions 3-10-5-5. In this figure, the responsiveness indicator is represented by the total weighted tardiness. This indicator, for a specific model, is calculated by dividing (the difference between the maximum obtained objective function in 100 iterations and the obtained objective function by the model in a specific iteration) by (the maximum obtained objective function in 100 iterations). Hence, a model with a greater responsiveness indicator is more satisfactory. To minimize the total weighted tardiness, we developed a new model that allows the “back-order”. The new formulation is presented in Appendix B.

Fig. 11 clearly shows that incorporating lot-streaming is an effective way to improve responsiveness in the ILSP so that the proposed model with lot-streaming had a 570% improvement on average compared to the model without lot-streaming. This volume of improvement compared to the improvement achieved in the solution quality indicator

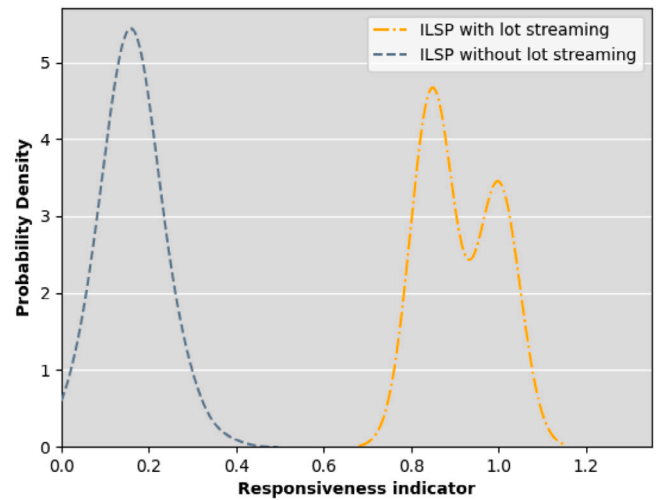


Fig. 11. The responsiveness indicator.

(7% according to Section 6.2.1.2) implies a much higher sensitivity of the time-dependent indicators to the lot-streaming compared to time-independent indicators.

6.2.1.5. Work-in-process. In manufacturing systems, buffers or storage between machines reduce the impact of disturbances and lack of synchronization in production. This buffer raises the work-in-process (WIP) inventory, which imposes additional costs on the system (Ma and Koren, 2004). In addition, allocating sufficient space for their storage is an operational challenge. Therefore, minimizing the work-in-process (WIP) inventory is always counted as one of the goals of manufacturers. In this section, the performance of the proposed model has been analyzed regarding the amount of WIP required to establish synchronization between two consecutive levels. To this end, a simulation is developed in which 100 randomly generated instances (with dimensions 3-10-5-5) are solved by ILSP with lot-streaming (which is described in Section 6.2.1.4) and the ILSP model without lot-streaming. Fig. 12 illustrates the KDE plot of the WIP indicator, which corresponds to the total work-in-process over the planning horizon. According to this figure, it can be seen that the inclusion of lot-streaming in the ILSP significantly reduces the WIP. The simulation results show an average improvement of 33% compared to ILSP without lot-streaming.

6.2.1.6. Machine idle time. Machine idle time is another indicator that can be economically significant. From an economic point of view, machine idleness can be counted as a kind of cost for production systems due to the loss of production opportunities. In this section, the performance of the proposed model in reducing the idle time of machines has been examined. For this purpose, 100 different randomly generated instances (with dimensions 3-10-5-5) are solved by the ILSP model with lot-streaming (the model presented in Section 6.2.1.4) and the ILSP model without lot-streaming (CTLSP). Fig. 13 is a KDE plot showing the total idle time of the machines during the planning horizon under 100 iterations of a simulation. This figure shows that the proposed lot-streaming model effectively reduces the machine's idle time. By using the proposed model, the average of the machine's idle time has been reduced from 1618 to 1123 time units (30% improvement).

6.2.1.6. Machine idle time. Analyzing the impact of four influential and controllable parameters on the objectives of the problem has been conducted in this subsection. These parameters are (i) demand ($D_{j,t}$), (ii) setup cost ($sc_{j,o,t}$), (iii) production cost ($pc_{j,o,t}$), and (iv) inventory holding cost ($hc_{j,o,t}$). In this context, we changed these parameters separately from -50% to +50% separately. To obtain more solid results, ten different instances with dimensions 5-6-5-6 are randomly generated,

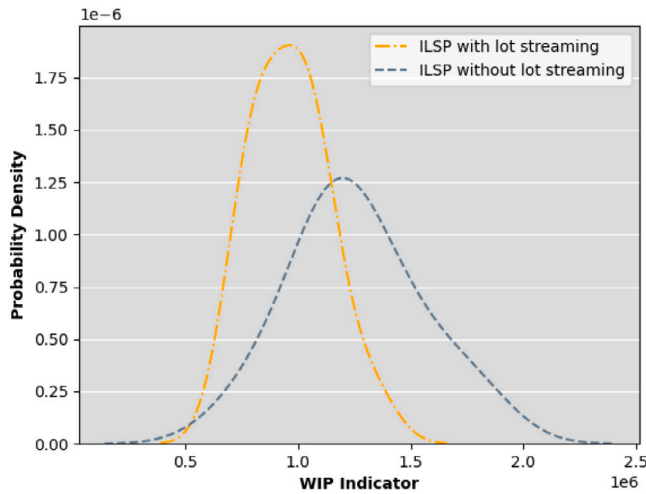


Fig. 12. The work-in-process indicator.

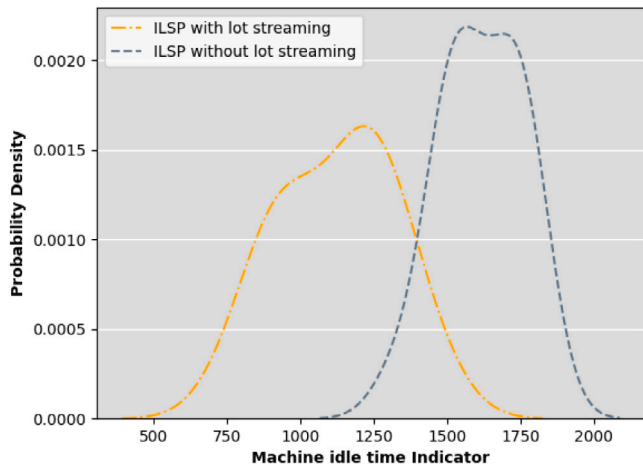


Fig. 13. The machine idle time indicator.

and the average changes of the objective function in ten instances are given in each case. The results presented in this subsection can provide useful insights for managers to shape their decisions to be prepared for similar changes. These results are presented in Fig. 14. As can be seen in this figure, objective value is more sensitive to changes in demand. Among the costs of the system, the production costs required to produce a unit of items have the greatest impact on the objective function. As a result, managers should pay more attention to this fact by mechanizing production processes or reducing material supply costs. In addition, the objective value is less sensitive to inventory holding cost, so the objective function remains almost unchanged when these costs increase above their original value. The reason is that the model tends to replace setup costs with inventory holding costs when $hc_{j,o,t}$ increases. In fact, the model prefers to reduce the total inventory holding costs by increasing the number of setups during the planning horizon. In this experiment, when $hc_{j,o,t}$ is increased by 50%, the total setup cost increases by 18% on average, while the total inventory holding cost decreases by 94% on average.

6.2.2. Analysis of the proposed algorithms

The results obtained by the proposed algorithms are summarized in Table 2. In this table, we analyzed the performance of the proposed algorithms for the data group ($\{\mathcal{T}\}$), with the tighter length of

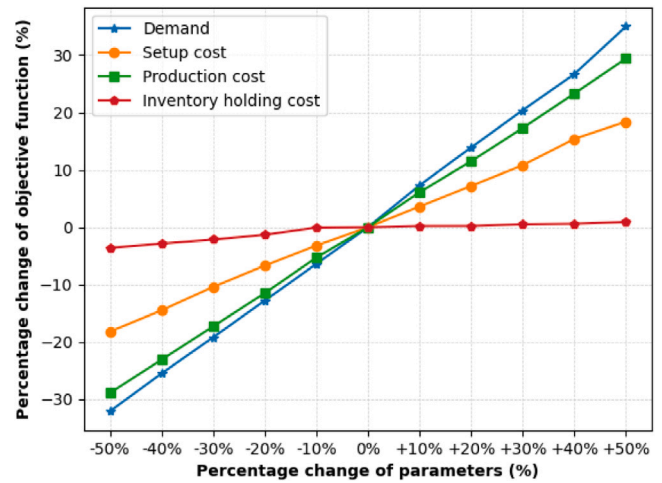


Fig. 14. Sensitivity analysis of model.

periods and capacity of the machines. A similar analysis for the data group ($\{\mathcal{W}\}$) is also presented in the supplementary material (<https://shorturl.at/IoD67>). We set the time limit to 3600 s for the solution methods.

By comparing the RPD, it can be observed that both the proposed solution methods can solve the problem effectively. In 6 instances out of 20, the final solutions obtained by FR-FO are equal to or better than the MIP model, while FR-FO solved them on average 7.6 times faster. This improvement is also impressively evident in the DH method. While DH's gap value from the MIP model is less than 1%, it was able to solve instances 433 times faster than the MIP model. In addition, these methods provided feasible solutions for the 12 instances that the MIP model cannot solve. Moreover, Table 2 shows that the DH method, with an average RPD close to zero, outperforms the FR-FO method, with an average RPD equal to 0.046. The preference of DH over FR-FO will become more apparent by comparing the CPU time. While FR-FO represents an average CPU time greater than 1947 s, the DH method provided an average CPU time of 263 s. We developed a lower bound (LB) to examine the quality of the results obtained by the proposed algorithms for instances with larger sizes. The developed LB is presented in Appendix C. The difference between the obtained solutions and the LB is shown with the GAP% in Table 2. Considering the values of the GAP% between the LB and the proposed heuristics, their defensible performance can be observed as the average values of the GAP%, which are equal to 6.4% and 2.1% for the FR-FO and DH, respectively.

The effectiveness of the feasibility and optimality cuts is a crucial factor for the performance of decomposition heuristics. In this regard, Fig. 15 depicts the obtained objective function during the solving procedure of the DH until reaching the first feasible solution for $\{\mathcal{T}\}$ 6-15-10-5. Accordingly, the ability of the DH to avoid repetitive solutions and its reasonable progress toward a feasible solution is evident as it is terminated after 15 iterations.

For the statistical comparison of the proposed solution methods, the individual value plot and interval plot with 95% confidence intervals (CIs) are employed for their optimality gap. The optimality gaps are obtained for 100 randomly generated instances with 5 machines, 5 periods, and a total number of operations between 30 to 40. Based on the values of the optimality gap, the corresponding plots are depicted in Fig. 16. Fig. 16(a) shows an individual value plot that visualizes the distribution spread and means of the values obtained by the algorithms. Fig. 16(a) illustrates that the DH achieved a more favorable distribution with a lower mean and variation. Moreover, in Fig. 16(a), the superiority of FR-FO over FR is evident. Concerning Fig. 16(b), the confidence

Table 2
Comparison of the proposed solution methods.

Total number of operations	Instances {T}J-O-M-T	Lower bound (LB)	Proposed model			FR-FO				DH			
			CPU time (s)	Objective function	RPD	CPU time (s)	Objective function	RPD	GAP%	CPU time (s)	Objective function	RPD	GAP%
12	{T}3-4-3-5	4001.0	4.1	4229.6	0.00	9.7	4229.6	0.00	5.7%	1.8	4237.6	0.002	5.9%
12	{T}4-3-3-6	5108.1	4.5	5448.27	0.00	7.4	5448.3	0.00	6.7%	2.1	5507.0	0.01	7.8%
30	{T}6-5-5-4	8213.1	>3600	8631.1	0.00	322.4	8670.0	0.005	5.6%	16.6	8931.8	0.03	8.8%
30	{T}3-10-5-5	8807.5	13.4	8841.19	0.00	70.5	8841.2	0.00	0.4%	2.8	8852.2	0.001	0.5%
30	{T}5-6-5-6	10236.7	>3600	10319.41	0.00	85.2	10319.4	0.00	0.8%	3.1	10338.0	0.002	1.0%
50	{T}5-10-5-4	10806.9	>3600	10815.55	0.00	291.6	10815.6	0.00	0.1%	4.7	10847.7	0.003	0.4%
50	{T}5-10-7-5	13756.1	>3600	13930.25	0.002	1415.7	13899.7	0.00	1.0%	1.1	13899.7	0.00	1.0%
50	{T}10-5-5-6	15680.7	>3600	15727.56	0.00	141.1	16619.4	0.06	6.0%	9.6	16346.7	0.04	4.2%
80	{T}10-8-5-4	18270.2	>3600	–	–	3562.3	18537.9	0.00	1.5%	68.7	18546.7	0.00	1.5%
80	{T}8-10-5-5	22019.4	>3600	–	–	1367.1	22707.8	0.02	3.1%	617.7	22182.4	0.00	0.7%
80	{T}20-4-4-6	29082.3	>3600	–	–	457.7	31688.4	0.08	9.0%	4.9	29214.7	0.00	0.5%
90	{T}6-15-10-5	22894.5	>3600	–	–	3155.5	23421.7	0.00	2.3%	291.7	23469.9	0.002	2.5%
100	{T}10-10-5-4	20653.0	>3600	–	–	>3600	21361.1	0.03	3.4%	235.7	20673.2	0.00	0.1%
100	{T}20-5-5-5	29266.1	>3600	–	–	2864.4	30746.6	0.05	5.1%	3.2	29403.6	0.00	0.5%
100	{T}5-20-10-6	30111.0	>3600	–	–	>3600	–	–	–	222.1	30236.1	0.00	0.4%
120	{T}10-12-8-5	34031.2	>3600	–	–	>3600	36900.7	0.07	8.4%	328.0	34569.5	0.00	1.6%
150	{T}10-15-10-4	31976.6	>3600	–	–	>3600	33216.3	0.02	3.9%	52.4	32454.7	0.00	1.5%
160	{T}10-16-10-6	50744.5	>3600	–	–	>3600	55725.9	0.10	9.8%	945.1	50810.1	0.00	0.1%
180	{T}12-15-10-5	47457.1	>3600	–	–	>3600	53537.4	0.11	12.8%	341.5	48091.9	0.00	1.3%
200	{T}10-20-15-4	44323.6	>3600	–	–	>3600	60472.7	0.34	36.4%	2118.4	45244.6	0.00	2.1%

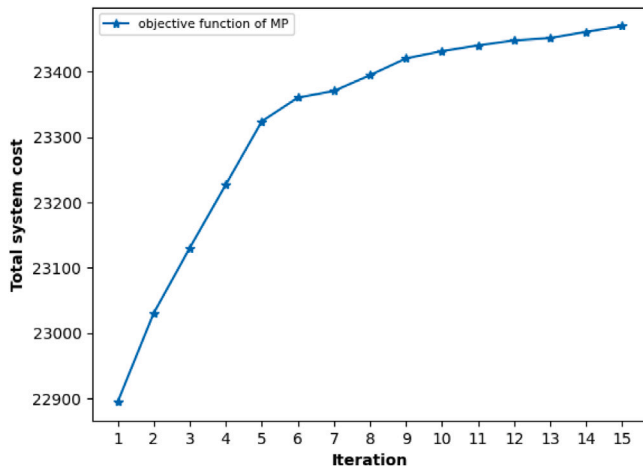


Fig. 15. Objective function changes through the DH solving procedure.

intervals do not overlap for any pair of solution methods. Hence, the means of the optimality gaps obtained by the solution methods could be significantly different.

7. Managerial insights

As a managerial insight, the proposed model showed that the lot-streaming efficiently reduces the system's operational cost by up to 7%. This is while the effectiveness is significantly increased in the time-oriented objective functions, such as total weighted tardiness. An average improvement of 570% in the total tardiness penalty is a significant improvement from a management perspective. On the other hand, given the rapidly changing market due to globalization, digitalization, and some unexpected disruptions in supply chains, such as the COVID-19 pandemic, production systems need to be dynamic and flexible to respond to these changes in a timely manner. This ability to recover the system quickly from disruptions is called system resilience (Alexopoulos et al., 2022). The large fluctuations in demand are a direct result of the recent dynamic market. In addition, the transition from mass production to customized products requires

the individualization of operations, which has reduced the knowledge about the processing time of operations. Thus, it increases the uncertainty in processing times. To investigate the system's resilience, we measured the re-schedulability indicator considering disruptions in demand and processing time. The experimental results show that the proposed model significantly improves this indicator by enhancing the synchronization between consecutive operations. In addition, the impact of the proposed model on the work-in-process (WIP) value has also been analyzed. WIP is a buffer between processes to avoid interference and lack of synchronization. WIP inventory is a challenge for management, both in terms of inventory cost and shop floor space occupancy. The proposed model reduces the WIP inventory by an average of 33% by increasing the synchronization between the different levels. The machine's idle time is another indicator that can be economically significant. From an economic point of view, machine idleness can be counted as a kind of lost opportunity cost for production systems. The experimental results show that the proposed lot-streaming equation integrated into the ILSP model reduces the machine idle time by 30% on average. Ojstersek et al. (2020) studied the impact of manufacturing flexibility on sustainability in three categories of impact: (i) environmental, (ii) social, and (iii) financial. Their results demonstrated a high degree of dependence between manufacturing flexibility and sustainability. Hence, it can be concluded that lot-streaming as an approach to increase flexibility in manufacturing can lead to an improvement in sustainability.

Although this paper discusses the various advantages of lot-streaming, it is not easy to exploit these advantages in all cases, for example, when lot integrity must be maintained for traceability reasons or when successive operations are located some distance apart. In addition, increasing synchronization through lot-streaming leads to higher costs, such as transportation costs (due to increased material handling between the two consecutive operations) or overhead costs (due to increased control and monitoring activities). Therefore, decision-makers need to consider the extent to which these cost increases are justifiable compared to reduced costs, such as the cost of lost-sales or shortages. From another perspective, according to the computational results, the proposed model has a more desirable performance in responding to uncertainty. Nevertheless, increasing synchronization reduces the safety level of the system and exposes it to more uncertainty. This requires constant monitoring and control of the processes, as well as more effective planning and measures to mitigate the production risks.

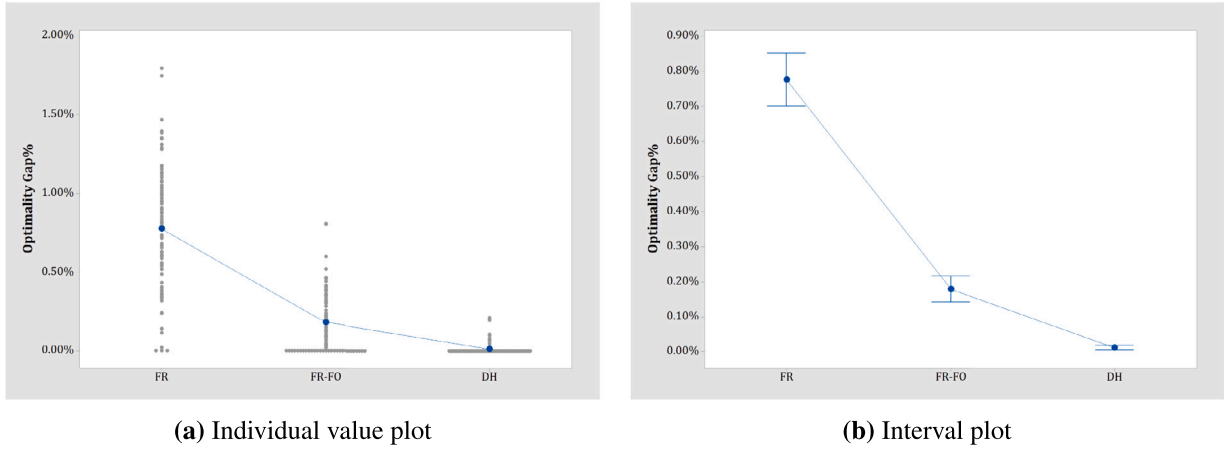


Fig. 16. Individual value plot and interval plot to compare the proposed solution methods.

8. Conclusions

This paper dealt with a simultaneous lot-sizing and job shop scheduling problem to minimize the total system cost. To achieve this goal, a novel mixed-integer linear programming model is developed that takes the lot-streaming in a big-bucket time frame into account. Computational results showed that the proposed model outperforms the small-bucket and big-bucket time models in the literature in six KPIs, including the solution quality, feasibility, re-schedulability, responsiveness, work-in-process, and machine idle time. Managerial insights are deduced according to the results obtained from these KPIs. In addition, to solve the medium- and large-sized problems, we proposed two efficient heuristics. First, the fix-and-relax algorithm as a constructive heuristic in combination with fix-and-optimize as an improvement heuristic (FR-FO). Different variants of the fix-and-relax heuristic first lead to an initial solution, which is further improved in the next step using a fix-and-optimize heuristic. Moreover, for the second heuristic, a powerful decomposition heuristic is proposed that solves the problem hierarchically in an iterative procedure. The lot-sizing and sequencing variables are determined in a master problem and the sub-problems, respectively, while the sub-problems are strengthened by constraint programming. A strong inequality based on the critical path is also proposed to add to the master problem as a feasibility cut and to deal with infeasible solutions. While both methods can provide high-quality solutions in most instances, the average CPU times of FR-FO and DH are 7.6 and 427 times faster than the MILP model, respectively.

Several promising avenues for application in different industries may extend the current study. An interesting line of research would be to extend the proposed decomposition heuristic for the model that considers sequence-dependent setup costs or time-based objective functions. Under these assumptions, due to the dependence of the objective function on the subproblem, it is necessary to develop an optimal cut. Another interesting line of research is to investigate the performance of the proposed decomposition heuristic in online and real-time decision-aiding support systems due to the low computational time achieved. Developing the model with carry-over or cross-over setup and machine reconfiguration are other fruitful directions for future research. Although the performance of the proposed model has been studied using the re-schedulability indicator under the uncertainty of demand and processing time, it can be extended to ensure the uncertainty of other parameters such as lead time, due date, machine availability, and so on. In addition, studying the applicability of the model for the different types of robust optimization or stochastic programming can be one of the fundamental directions for future studies.

Data availability

We have shared the link of required data in the manuscript file.

Acknowledgments

The authors would like to thank the Editor-in-Chief and autonomous reviewers for their valuable comments, which greatly improved the quality of this manuscript. This research has been supported by the Ministry of Education, Youth and Sports within the dedicated program ERC CZ under the project POSTMAN with reference LL1902 and the Grant Agency of the Czech Republic under the Project GACR 22-31670S.

Appendix A. Multi setup

As mentioned in the paper, constraints (11) and (12) are valid for at most one setup per operation in a period. Hence Theorem A.1 is presented here to ensure in the optimal solution; there is at most one setup for each operation per period.

Theorem A.1. *In the optimal solution of the proposed model, there is at most one setup for each operation per period.*

Proof. To prove it, first, we assume solution χ is an optimum solution with the setup of U times for $O_{j,o}$ on the machine m in the period t . Solution χ' is obtained by integrating all setups of $O_{j,o}$ into the first setup. $ST_{m,t}(S)$ is the total setup time on machine m in period t for solution S . Therefore:

$$ST_{m,t}(\chi') = ST_{m,t}(\chi) - \sum_{u=2}^U \sum_{(j',o') \in U} (\delta_{j',o',j,o} \cdot y_{u,j,o,t}^{j',o'} + \delta_{j,o,j',o'} \cdot y_{j,o,u,t}^{j',o'}) + \sum_{u=2}^U \sum_{(j',o') \in U} \sum_{(j'',o'') \in U} \delta_{j',o',j'',o''} \cdot y_{u,j,o,t}^{j',o'} \cdot y_{j,o,u,t}^{j'',o''}$$

while:

$$y_{j,o,u,t}^{j',o'} = \begin{cases} 1 & \text{iff } O_{j,o} \text{ produced immediately before } O_{j',o'} \text{ at its } u\text{th setup in period } t; 0, \text{ otherwise} \\ y_{u,j,o,t}^{j',o'} = \begin{cases} 1 & \text{iff } O_{j,o} \text{ produced immediately after } O_{j',o'} \text{ at its } u\text{th setup in period } t; 0, \text{ otherwise} \end{cases} \end{cases}$$

Because of the triangle inequality, the following relationship is always valid.

$$\sum_{u=2}^U \sum_{(j',o') \in U} (\delta_{j',o',j,o} \cdot y_{u,j,o,t}^{j',o'} + \delta_{j,o,j',o'} \cdot y_{j,o,u,t}^{j',o'}) \geq \sum_{u=2}^U \sum_{(j',o') \in U} \sum_{(j'',o'') \in U} \delta_{j',o',j'',o''} \cdot y_{u,j,o,t}^{j',o'} \cdot y_{j,o,u,t}^{j'',o''}$$

Therefore $ST_{m,t}(\chi') \leq ST_{m,t}(\chi)$ and χ' is also feasible according to constraints (7) and (12). Forasmuch as the objective function of χ' is $(U - 1) \cdot sc_{j,o,t}$ units smaller than the objective function of χ , the optimality assumption of χ is violated, and the proof is completed. $\square$

Appendix B. The model with total weighted tardiness objective function

New parameter

$TP_{j,t}$ Tardiness penalty per time unit for the demand of job j in at the end of period t

New Variables

$Tr_{j,t}$ Tardiness for the demand of job j in period t
 $Sh_{j,t}$ Shortage for the demand of job j at the end of period t
 $\pi_{j,t}$ 1 iff job j is subject of shortage at the end of period t ;
 0, otherwise

$$\text{Min} \sum_{t \in T} \sum_{j \in J} Tr_{j,t} \cdot TP_{j,t} \quad (40)$$

Constraints (3), (4), (5), (6), (7), (8), (9), (10), (11), and (12)

$$I_{j,h_{j,t-1}} - Sh_{j,t-1} + x_{j,h_{j,t}} - I_{j,h_{j,t}} + Sh_{j,t} = D_{j,t} \quad \forall t, \forall j \quad (41)$$

$$x_{j,h_{j,t}} \geq Sh_{j,t-1} \quad \forall t, \forall j \quad (42)$$

$$I_{j,h_{j,t}} \leq x_{j,h_{j,t}} - Sh_{j,t-1} \quad \forall t, \forall j \quad (43)$$

$$Sh_{j,t} - \pi_{j,t} \cdot G \leq 0 \quad \forall t, \forall j \quad (44)$$

$$Tr_{j,t} \geq s_{j,o,(t+1)} + x_{j,h_{j,t}} \cdot p_{j,o} - t \cdot L - (1 - \pi_{j,t}) \cdot G \quad \forall t, \forall j \quad (45)$$

$$\{Tr_{j,t}, Sh_{j,t}\} \in \mathbb{R}^+, \pi_{j,t} \in \{0, 1\} \quad \forall t, \forall j \quad (46)$$

The objective function (40) is the weighted sum of tardiness. Constraint (41) ensures the material balance between two consecutive periods instead of Constraint (2). Constraints (42) and (43) are two new constraints that ensure all shortages must be delivered in the next period. Moreover, the Constraints (44) and (45) are added to the model to determine the amount of tardiness related to the demand of job j in period t . Moreover, to compensate for the shortage in the last period of the planning horizon, we considered one additional period with zero demand at the end of the planning horizon.

Appendix C. A lower bound of the problem

A lower bound is extended to examine the quality of the solutions obtained by the proposed heuristics. This lower bound is a relaxed version of the original model, which is computationally straightforward. Similar to the master problem (Section 5.2.1), in the proposed lower bound, variables $x_{j,o,t}$, $z_{j,o,t}$, and $I_{j,o,t}$ are determined, neglecting the sequence variables. Calculating the maximum completion time of the operations in a period and restricting it to the length of the period is a valid and strong inequality that can improve the lower bound. The simplest lower bound for the maximum completion time is the total processing time of operations assigned to each machine. However, providing a tighter inequality is more beneficial. Proposition 1 defines a valid inequality for the maximum completion time adapted from Carlier and Pinson (1989). In addition to a lower bound for the setup times, this inequality compounds the minimum possible release and tail times to the total processing time of operations assigned to a machine. The release time of an operation represents its earliest possible starting time, and the tail time specifies the lowest possible completion time of its produced successor operations of the corresponding job. Before expressing Proposition 1, it is necessary to introduce zf_{jot} as a binary variable that is equal to 1 if O_{jo} is processed as the first operation on its eligible machine in period t .

Proposition 1. LB_t is a lower bound for the maximum completion time of the operations in period t .

$$LB_t = \max_m \{H_{mt}\} \quad (47)$$

Where:

$$\begin{aligned} H_{mt} = & \min_{\substack{(j,o) \\ a_{jo}=m}} \left\{ s_{jot} + G \cdot (1 - z_{jot}) \right\} + \sum_{\substack{(j,o) \\ a_{jo}=m}} x_{j,o,t} \cdot p_{j,o} \\ & + \min_{\substack{(j',o') \\ a_{j'o'}=m}} \left\{ \delta_{j',o',j,o} \right\} \cdot (z_{jot} - zf_{jot}) \\ & + \min_{\substack{(j,o) \\ a_{jo}=m}} \left\{ \max_{O_{j,o} \in Q_{j,o'}} \left\{ s_{jot} + x_{j,o',t} \cdot p_{j,o'} \right\} + G \cdot (1 - z_{jot}) \right\} \end{aligned} \quad (48)$$

$$\sum_{\substack{(j,o) \\ a_{jo}=m}} zf_{jot} = 1 \quad \forall t, \forall m \quad (49)$$

$$zf_{jot} \leq z_{jot} \quad \forall t, \forall (j, o) \quad (50)$$

Proof. The minimum time required to start processing a machine equals the minimum release time of the operations assigned to the machine. Therefore it can be added to the total processing time of operations on the machine. The release time of operation O_{jo} in period t can be represented by s_{jot} while it is calculated based on the precedence constraints (constraints (9), (10), (11), and (12)) with its preceding operations. Hence, the minimum release time of operations assigned to machine m is:

$$\min_{\substack{(j,o) \\ a_{jo}=m}} \{s_{jot}\}$$

On the other hand, the maximum completion time cannot be less than the finish time of the last operation on a machine plus its tail time. The tail time of operation O_{jo} in period t can be represented by $\max_{\substack{(j,o') \\ O_{j,o'} \in Q_{j,o'}}} \{s_{jot} + x_{j,o',t} \cdot p_{j,o'}\}$ while it is calculated based on the precedence constraints with its successor operations. The minimum tail time of the operations assigned to a machine m provides a lower bound for the tail time of the last operation on machine m as follows:

$$\min_{\substack{(j,o) \\ a_{jo}=m}} \left\{ \max_{O_{j,o'} \in Q_{j,o'}} \{s_{jot} + x_{j,o',t} \cdot p_{j,o'}\} \right\}$$

Therefore, for tightening the lower bound of the maximum completion time, the minimum release and tail times for the operations assigned to the machine can be added to the total processing times of that operations. $\square$

According to Proposition 1 the constraint (51) is added to the MP.

$$LB_t \leq L \quad (51)$$

Constraint (48) makes the lower bound model a nonlinear integer programming model. Therefore, the auxiliary variables σ_{mt}^1 and σ_{mt}^2 are defined. These continuous variables linearize two minimum functions of each machine's release time and tail time, respectively. The following constraints are added to the lower bound model instead of constraint (48).

$$\sigma_{mt}^1 \geq s_{jot} - G \cdot (1 - q_{jot}^1) \quad \forall t, \forall m, \forall (j, o) \mid a_{jo} = m \quad (52)$$

$$\sigma_{mt}^2 \geq \theta_{jot} - G \cdot (1 - q_{jot}^2) \quad \forall t, \forall m, \forall (j, o) \mid a_{jo} = m \quad (53)$$

$$q_{jot}^h \leq z_{jot} \quad \forall t, \forall m, \forall (j, o), h = 1, 2 \mid a_{jo} = m \quad (54)$$

$$\theta_{jot} \geq s_{jot} + x_{j,o',t} \cdot p_{j,o'} \quad \forall t, \forall (j, o, o') \mid O_{j,o} \in Q_{j,o'} \quad (55)$$

$$L \geq \sigma_{mt}^1 + \sum_{(j,o)} x_{j,o,t} \cdot p_{j,o} + \min_{a_{j,o'}=m} \left\{ \delta_{j',o',j,o} \right\} \cdot (z_{jot} - z_{fjot}) + \sigma_{mt}^2 \quad \forall t, \forall m \quad (56)$$

$$\{s_{jot}, x_{j,o,t}, \sigma_{mt}^1, \sigma_{mt}^2, \theta_{jot}\} \in \mathbb{R}^+ \quad \{z_{jot}, z_{fjot}, q_{jot}^1, q_{jot}^2\} \in \{0, 1\}$$

$$\forall t, \forall m, \forall (j, o)$$

Constraints (52) to (56) are a linearized version of the lower bound presented in (48). q_{jot}^1 and q_{jot}^2 are binary variables while specifying which operations have the minimum release and tail time on machine m , respectively. θ_{jot} is a variable that represent $\max_{\substack{(j,o') \\ O_{j,o'} \in O_{j,o}}} \{s_{j,o',t} + x_{j,o',t} \cdot p_{j,o'}\}$ in the lower bound presented in Proposition 1.

References

- Alexopoulos, Kosmas, Anagiannis, Ioannis, Nikolakis, Nikolaos, Chrysosolouris, George, 2022. A quantitative approach to resilience in manufacturing systems. *Int. J. Prod. Res.* 60 (24), 7178–7193.
- Almada-Lobo, Bernardo, Klabjan, Diego, Antónia Carravilla, Maria, Oliveira, José F., 2007. Single machine multi-product capacitated lot sizing with sequence-dependent setups. *Int. J. Prod. Res.* 45 (20), 4873–4894.
- Almeder, Christian, Klabjan, Diego, Traxler, Renate, Almada-Lobo, Bernardo, 2015. Lead time considerations for the multi-level capacitated lot-sizing problem. *European J. Oper. Res.* 241 (3), 727–738.
- Andriolo, Alessandro, Battini, Daria, Grubbström, Robert W., Persona, Alessandro, Sgarbossa, Fabio, 2014. A century of evolution from harris basic lot size model: Survey and research agenda. *Int. J. Prod. Econ.* 155, 16–38.
- Babaei, Mohammad, Mohammadi, Mohammad, Ghomi, S.M.T. Fatemi, 2014. A genetic algorithm for the simultaneous lot sizing and scheduling problem in capacitated flow shop with complex setups and backlogging. *Int. J. Adv. Manuf. Technol.* 70 (1–4), 125–134.
- Babaei, M., Mohammadi, M., Ghomi, S.M.T.F., Sobhanallahi, M., 2012. Two parameter-tuned metaheuristic algorithms for the multi-level lot sizing and scheduling problem. *Int. J. Ind. Eng. Comput.* 3 (5), 751–766.
- Baldo, Tamara A., Santos, Maristela O., Almada-Lobo, Bernardo, Morabito, Reinaldo, 2014. An optimization approach for the lot sizing and scheduling problem in the brewery industry. *Comput. Ind. Eng.* 72, 58–71.
- Barzanji, Ramin, Naderi, Bahman, Begen, Mehmet A., 2020. Decomposition algorithms for the integrated process planning and scheduling problem. *Omega* 93, 102025.
- Brüggemann, Wolfgang, Jahnke, Hermann, 1994. DLSP for two-stage multi-item batch production. *Int. J. Prod. Res.* 32 (4), 755–768.
- Brüggemann, Wolfgang, Jahnke, Hermann, 2000. The discrete lot-sizing and scheduling problem: Complexity and modification for batch availability. *European J. Oper. Res.* 124 (3), 511–528.
- Carlier, Jacques, Pinson, Éric, 1989. An algorithm for solving the job-shop problem. *Manage. Sci.* 35 (2), 164–176.
- Carvalho, Desiree M., Nascimento, Mariá C.V., 2021. Hybrid metaheuristics to solve the integrated lot sizing and scheduling problem on parallel machines with sequence-dependent and non-triangular setup. *European J. Oper. Res.*
- Castro, Pedro M., Grossmann, Ignacio E., 2012. Generalized disjunctive programming as a systematic modeling framework to derive scheduling formulations. *Ind. Eng. Chem. Res.* 51 (16), 5781–5792.
- Claassen, G.D.H., Gerdessen, Johanna C., Hendrix, Eligius M.T., van der Vorst, Jack G.A.J., 2016. On production planning and scheduling in food processing industry: Modelling non-triangular setups and product decay. *Comput. Oper. Res.* 76, 147–154.
- Clark, Alistair R., 2003. Optimization approximations for capacity constrained material requirements planning. *Int. J. Prod. Econ.* 84 (2), 115–131.
- Dauzère-Pérès, Stéphane, Lasserre, Jean-Bernard, 2002. On the importance of sequencing decisions in production planning and scheduling. *Int. Trans. Oper. Res.* 9 (6), 779–793.
- de Araujo, Silvio A., Arenales, Marcos N., Clark, Alistair R., 2007. Joint rolling-horizon scheduling of materials processing and lot-sizing with sequence-dependent setups. *J. Heuristics* 13 (4), 337–358.
- Drexel, Andreas, Kimms, Alf, 1997. Lot sizing and scheduling—survey and extensions. *European J. Oper. Res.* 99 (2), 221–235.
- Fandel, Günter, Stammen-Hegene, Cathrin, 2006. Simultaneous lot sizing and scheduling for multi-product multi-level production. *Int. J. Prod. Econ.* 104 (2), 308–316.
- Filho, Márcio A.F. Belo, Santos, Maristela O., Meneses, Cláudio N., 2012. Asynchronous teams for joint lot-sizing and scheduling problem in flow shops. *Int. J. Prod. Res.* 50 (20), 5809–5822.
- Fleischmann, Bernhard, 1990. The discrete lot-sizing and scheduling problem. *European J. Oper. Res.* 44 (3), 337–348.
- Fleischmann, Bernhard, 1994. The discrete lot-sizing and scheduling problem with sequence-dependent setup costs. *European J. Oper. Res.* 75 (2), 395–404.
- Fleischmann, Bernhard, Meyr, Herbert, 1997. The general lotsizing and scheduling problem. *Oper.-Res.-Spektrum* 19 (1), 11–21.
- Florian, Michael, Lenstra, Jan Karel, Rinnooy Kan, A.H.G., 1980. Deterministic production planning: Algorithms and complexity. *Manage. Sci.* 26 (7), 669–679.
- Giglio, Davide, Paolucci, Massimo, Roshani, Abdolreza, 2017. Integrated lot sizing and energy-efficient job shop scheduling problem in manufacturing/remanufacturing systems. *J. Clean. Prod.* 148, 624–641.
- Gomez Urrutia, Edwin David, Aggoune, Riad, Dauzère-Pérès, Stéphane, 2014. Solving the integrated lot-sizing and job-shop scheduling problem. *Int. J. Prod. Res.* 52 (17), 5236–5254.
- Göthe-Lundgren, Maud, Lundgren, Jan T., Persson, Jan A., 2002. An optimization model for refinery production scheduling. *Int. J. Prod. Econ.* 78 (3), 255–270.
- Grünert, Tore, 1998. Multi-Level Sequence Dependent Dynamic Lotsizing and Scheduling. Shaker.
- Haase, Knut, 1996. Capacitated lot-sizing with sequence dependent setup costs. *Oper.-Res.-Spektrum* 18 (1), 51–59.
- Hindi, K.S., 1995. Solving the single-item, capacitated dynamic lot-sizing problem with startup and reservation costs by tabu search. *Comput. Ind. Eng.* 28 (4), 701–707.
- Hooker, John N., 2007. Planning and scheduling by logic-based Benders decomposition. *Oper. Res.* 55 (3), 588–602.
- Huang, Rong-Hwa, 2010. Multi-objective job-shop scheduling with lot-splitting production. *Int. J. Prod. Econ.* 124 (1), 206–213.
- Kaczmarczyk, Waldemar, 2011. Proportional lot-sizing and scheduling problem with identical parallel machines. *Int. J. Prod. Res.* 49 (9), 2605–2623.
- Karimi-Nasab, M., Modarres, Mohammad, Seyedhoseini, S.M., 2015. A self-adaptive PSO for joint lot sizing and job shop scheduling with compressible process times. *Appl. Soft Comput.* 27, 137–147.
- Karimi-Nasab, Mehdi, Seyedhoseini, S.M., 2013. Multi-level lot sizing and job shop scheduling with compressible process times: A cutting plane approach. *European J. Oper. Res.* 231 (3), 598–616.
- Kimms, A., 1996. Competitive methods for multi-level lot sizing and scheduling: Tabu search and randomized regrets. *Int. J. Prod. Res.* 34 (8), 2279–2298.
- Kimms, Alf, 1999. A genetic algorithm for multi-level, multi-machine lot sizing and scheduling. *Comput. Oper. Res.* 26 (8), 829–848.
- Kimms, Alf, 2012. Multi-Level Lot Sizing and Scheduling: Methods for Capacitated, Dynamic, and Deterministic Models. Springer Science & Business Media.
- Kimms, Alf, Drexl, Andreas, 1998. Proportional lot sizing and scheduling: Some extensions. *Netw. Int. J.* 32 (2), 85–101.
- Lang, Jan Christian, Shen, Zuo-Jun Max, 2011. Fix-and-optimize heuristics for capacitated lot-sizing with sequence-dependent setups and substitutions. *European J. Oper. Res.* 214 (3), 595–605.
- Li, Yongjian, Cai, Xiaoqiang, Xu, Lei, Yang, Wenxia, 2016. Heuristic approach on dynamic lot-sizing model for durable products with end-of-use constraints. *Ann. Oper. Res.* 242 (2), 265–283.
- Ma, Yuan-Hung Kevin, Koren, Yoram, 2004. Operation of manufacturing systems with work-in-process inventory and production control. *CIRP Ann.* 53 (1), 361–365.
- Maes, Johan, McClain, John O., Van Wassenhove, Luk N., 1991. Multilevel capacitated lotsizing complexity and LP-based heuristics. *European J. Oper. Res.* 53 (2), 131–148.
- Mahdiah, Masoumeh, Bijari, Mehdi, Clark, Alistair, 2011. Simultaneous lot sizing and scheduling in a flexible flow line. *J. Ind. Syst. Eng.* 5 (2), 107–119.
- Meyr, Herbert, 2000. Simultaneous lotsizing and scheduling by combining local search with dual reoptimization. *European J. Oper. Res.* 120 (2), 311–326.
- Muramatsu, Kenji, Warman, Aditya, Kobayashi, Minoru, 2003. A near-optimal solution method of multi-item multi-process dynamic lot size scheduling problem. *JSME Int. J. C* 46 (1), 46–53.
- Ojstersek, R., Acko, B., Buchmeister, B., 2020. Simulation study of a flexible manufacturing system regarding sustainability. *Int. J. Simul. Model.* 19 (1), 65–76.
- Ouenniche, Jamal, Bector, F., 1998. Sequencing, lot sizing and scheduling of several products in job shops: the common cycle approach. *Int. J. Prod. Res.* 36 (4), 1125–1140.
- Poursabzi, O., Mohammadi, M., Naderi, B., 2018. An improved model and a heuristic for capacitated lot sizing and scheduling in job shop problems. *Sci. Iran.* 25 (6), 3667–3684.
- Qin, Wei, Zhuang, Zilong, Liu, Yang, Tang, Ou, 2019. A two-stage ant colony algorithm for hybrid flow shop scheduling with lot sizing and calendar constraints in printed circuit board assembly. *Comput. Ind. Eng.* 138, 106115.
- Ramezani, Reza, Saidi-Mehrabad, Mohammad, Fattahi, Parviz, 2013a. Integrated lot-sizing and scheduling with overlapping for multi-level capacitated production system. *Int. J. Comput. Integr. Manuf.* 26 (7), 681–695.
- Ramezani, Reza, Saidi-Mehrabad, Mohammad, Fattahi, Parviz, 2013b. MIP formulation and heuristics for multi-stage capacitated lot-sizing and scheduling problem with availability constraints. *J. Manuf. Syst.* 32 (2), 392–401.
- Ramezani, Reza, Saidi-Mehrabad, Mohammad, Teimoury, Ebrahim, 2013c. A mathematical model for integrating lot-sizing and scheduling problem in capacitated flow shop environments. *Int. J. Adv. Manuf. Technol.* 66 (1–4), 347–361.
- Ríos-Solís, Yasmín Á., Ibarra-Rojas, Omar J., Cabo, Marta, Possani, Edgar, 2020. A heuristic based on mathematical programming for a lot-sizing and scheduling problem in mold-injection production. *European J. Oper. Res.* 284 (3), 861–873.

- Rodoplu, Melek, Arbaoui, Taha, Yalaoui, Alice, 2020. A fix-and-relax heuristic for the single-item lot-sizing problem with a flow-shop system and energy constraints. *Int. J. Prod. Res.* 58 (21), 6532–6552.
- Rohaninejad, Mohammad, Kheirikhah, Amirsaman, Fattahi, Parviz, 2015. Simultaneous lot-sizing and scheduling in flexible job shop problems. *Int. J. Adv. Manuf. Technol.* 78 (1–4), 1–18.
- Rohaninejad, Mohammad, Sahraeian, Rashed, Nouri, Behdin Vahedi, 2016. Multi-objective optimization of integrated lot-sizing and scheduling problem in flexible job shops. *RAIRO Oper. Res.* 50 (3), 587–609.
- Sahling, Florian, Buschkühl, Lisbeth, Tempelmeier, Horst, Helber, Stefan, 2009. Solving a multi-level capacitated lot sizing problem with multi-period setup carry-over via a fix-and-optimize heuristic. *Comput. Oper. Res.* 36 (9), 2546–2553.
- Salido, Miguel A., Escamilla, Joan, Barber, Federico, Giret, Adriana, 2017. Rescheduling in job-shop problems for sustainable manufacturing systems. *J. Clean. Prod.* 162, S121–S132.
- Seeanner, Florian, Almada-Lobo, Bernardo, Meyr, Herbert, 2013. Combining the principles of variable neighborhood decomposition search and the fix&optimize heuristic to solve multi-level lot-sizing and scheduling problems. *Comput. Oper. Res.* 40 (1), 303–317.
- Seeanner, Florian, Meyr, Herbert, 2013. Multi-stage simultaneous lot-sizing and scheduling for flow line production. *OR Spectrum* 35 (1), 33–73.
- Stadtler, Hartmut, 2011. Multi-level single machine lot-sizing and scheduling with zero lead times. *European J. Oper. Res.* 209 (3), 241–252.
- Stadtler, Hartmut, Sahling, Florian, 2013. A lot-sizing and scheduling model for multi-stage flow lines with zero lead times. *European J. Oper. Res.* 225 (3), 404–419.
- Toledo, Claudio Fabiano Motta, da Silva Arantes, Márcio, Hossomi, Marcelo Yukio Bressan, França, Paulo Morelato, Akartunali, Kerem, 2015. A relax-and-fix with fix-and-optimize heuristic applied to multi-level lot-sizing problems. *J. Heuristics* 21 (5), 687–717.
- Wan, Guohua, Zhan, Yang, 2021. Multi-level, multi-stage lot-sizing and scheduling in the flexible flow shop with demand information updating. *Int. Trans. Oper. Res.* 28 (4), 2191–2217.
- Wand, Matt P., Jones, M. Chris, 1994. *Kernel Smoothing*. CRC Press.
- Wolosewicz, Cathy, Dauzère-Pérès, Stéphane, Aggoune, Riad, 2015. A Lagrangian heuristic for an integrated lot-sizing and fixed scheduling problem. *European J. Oper. Res.* 244 (1), 3–12.
- Xiao, Jing, Yang, Huasheng, Zhang, Canrong, Zheng, Li, Gupta, Jatinder N.D., 2015. A hybrid Lagrangian-simulated annealing-based heuristic for the parallel-machine capacitated lot-sizing and scheduling problem with sequence-dependent setup times. *Comput. Oper. Res.* 63, 72–82.
- Xiao, Jing, Zhang, Canrong, Zheng, Li, Gupta, Jatinder N.D., 2013. MIP-based fix-and-optimize algorithms for the parallel machine capacitated lot-sizing and scheduling problem. *Int. J. Prod. Res.* 51 (16), 5011–5028.
- Yan, Hong-Sen, Xia, Qi-Feng, Zhu, Min-Ru, Liu, Xia-Ling, Guo, Zhi-Min, 2003. Integrated production planning and scheduling on automobile assembly lines. *IIE Trans.* 35 (8), 711–725.